

① Interductary Concepts:-

② Software Development life cycle (SDLC):-

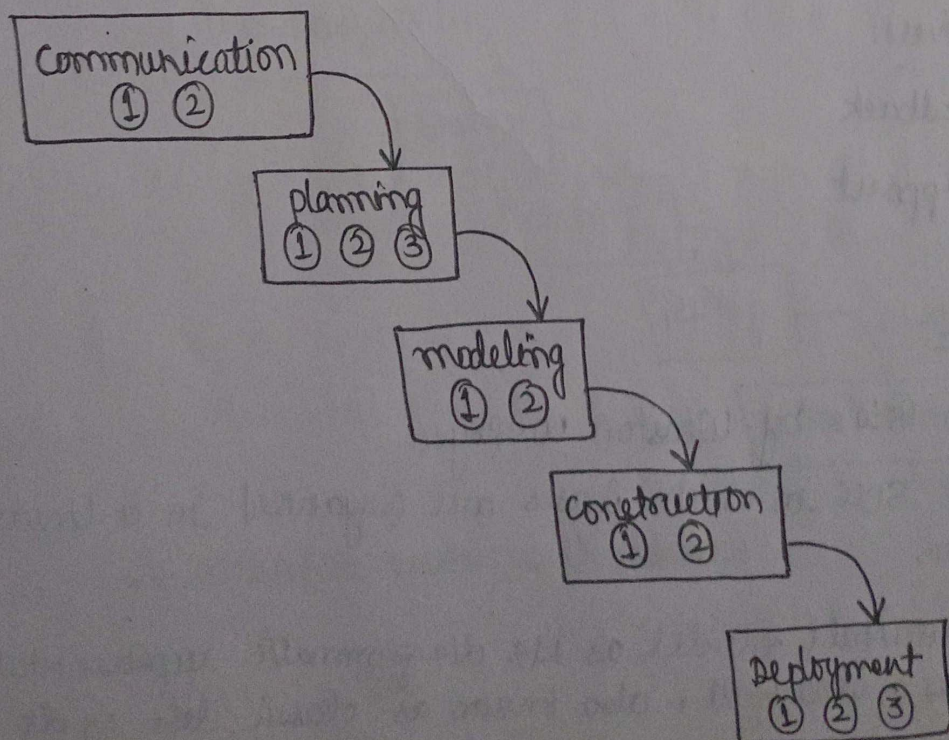
③ Software development organization follows some process when developing a software product, in mature organization this is well defined and manage. In SDLC we develop software in a systematic and disciplined manner.

→ SDLC well define entry and exit for every stage. It makes assessment possible, time prediction, cost prediction, scheduling, to identify faults early.

→ Selection of correct development model play an important role in cost, quality, overall success of the project.

④ Traditional Models-

⑤ The traditional Model in software engineering refers to structured, sequential and predefined approaches to software development.



1. Communication

- ↳ project initiation.
- ↳ Requirement gathering

2. planning

- ↳ estimation
- ↳ scheduling
- ↳ tracking

3. Modeling

- ↳ analysis
- ↳ design

4. construction

- ↳ code
- ↳ test

5. Deployment

- ↳ deliver.
- ↳ Feedback
- ↳ Support

⑥ Waterfall Model:-

⑥ Developed in the 1970s by Winston W. Royce.

→ It is a simplest SDLC in which phases are organised in a linear and sequential order.

→ It is called waterfall model as its diagrammatic representation resembles as like a waterfall, also known as classic life cycle model

→ This type of model is basically used for the small to medium-sized projects with clear, well-defined requirements.

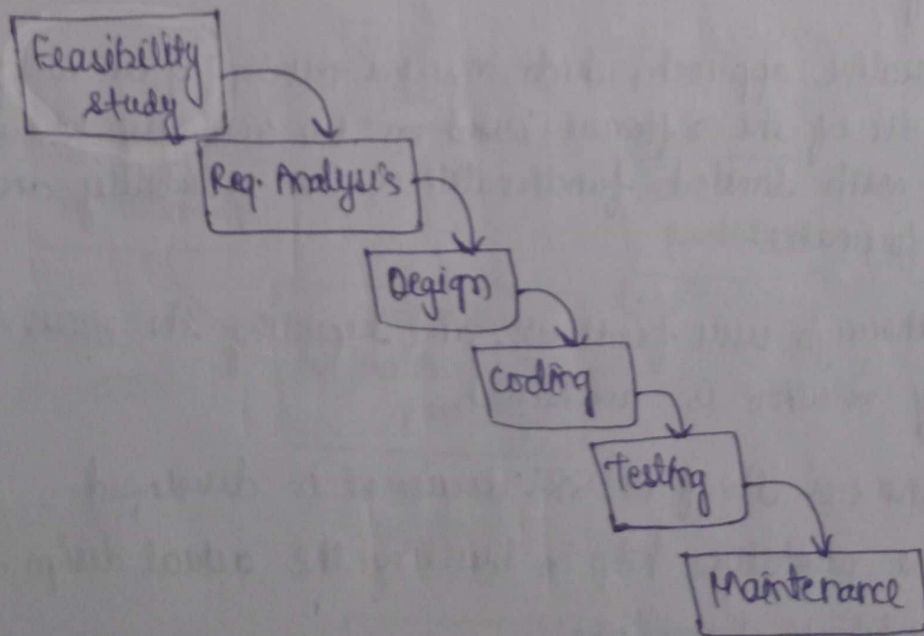
→ projects where minimal changes are expected during the development process,

① Advantages:-

1. Easy to understand.
2. Each phase has well defined input and output, phases are processed and completed one at a time.
3. Low cost and easy to schedule.

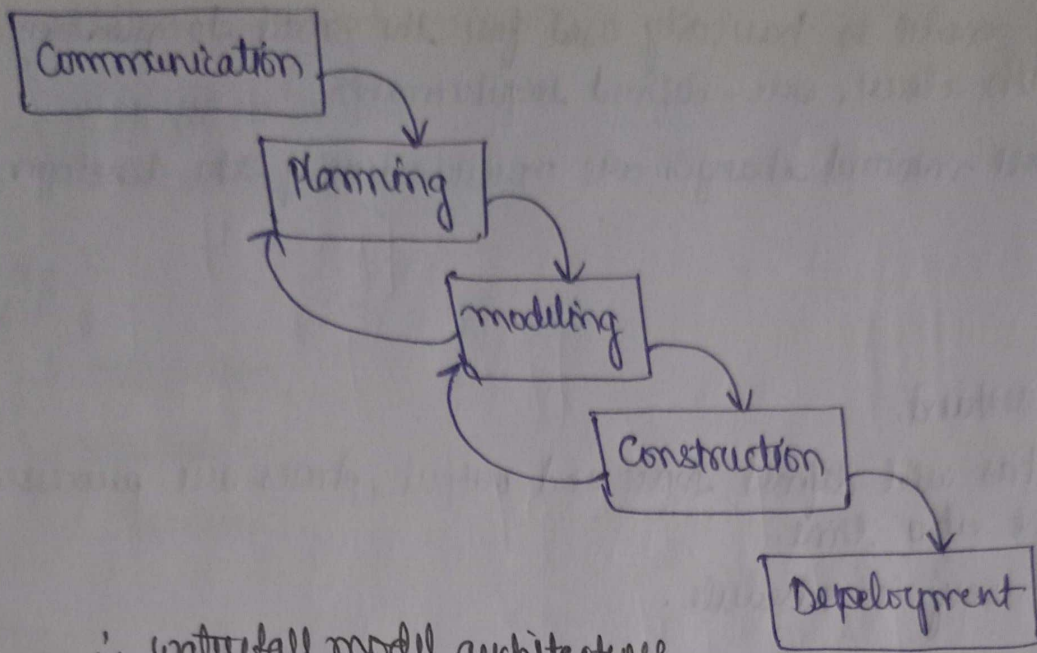
② Disadvantage:-

1. Not suitable to accommodate any change or iteration once development has begun.
2. Not good for large size projects.
3. High amount of risk and uncertainty.



∴ waterfall model architecture.





∴ waterfall model architecture

② Prototype Model

② Most of the customers are not sure about the functionality they require from the software, as a result the final software is not according to exact demand.

→ It is an iterative approach, which involves developing an early working model of the software based on the currently known requirements with limited functionalities, low reliability and untuned performance.

→ Refining it through user feedback, and repeating the process until a satisfactory solution is achieved.

→ After this process the final SRS document is developed.

→ Developing the prototype help in building the actual design.

② prototype model is of two type:-

(i) Evolutionary prototype.

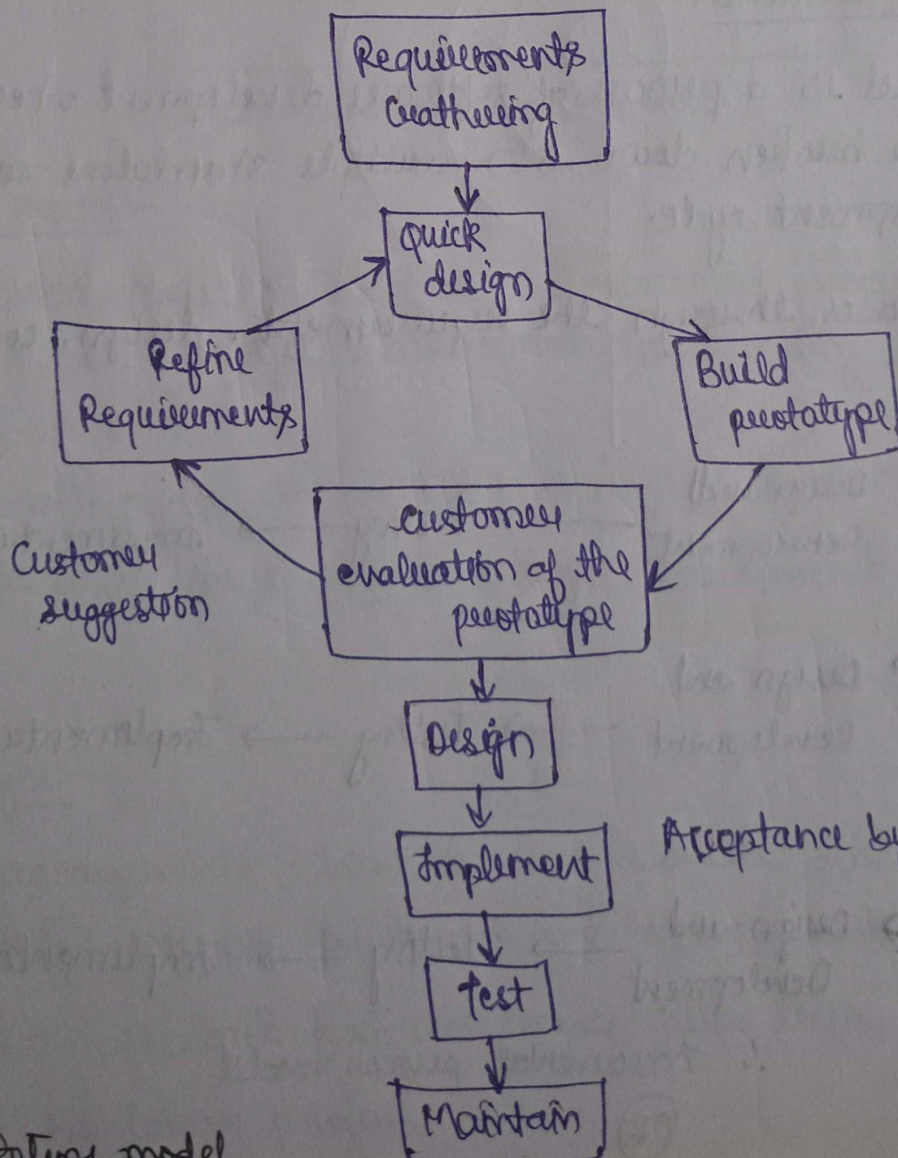
(ii) Throwaway prototype.

⑤ Advantage:-

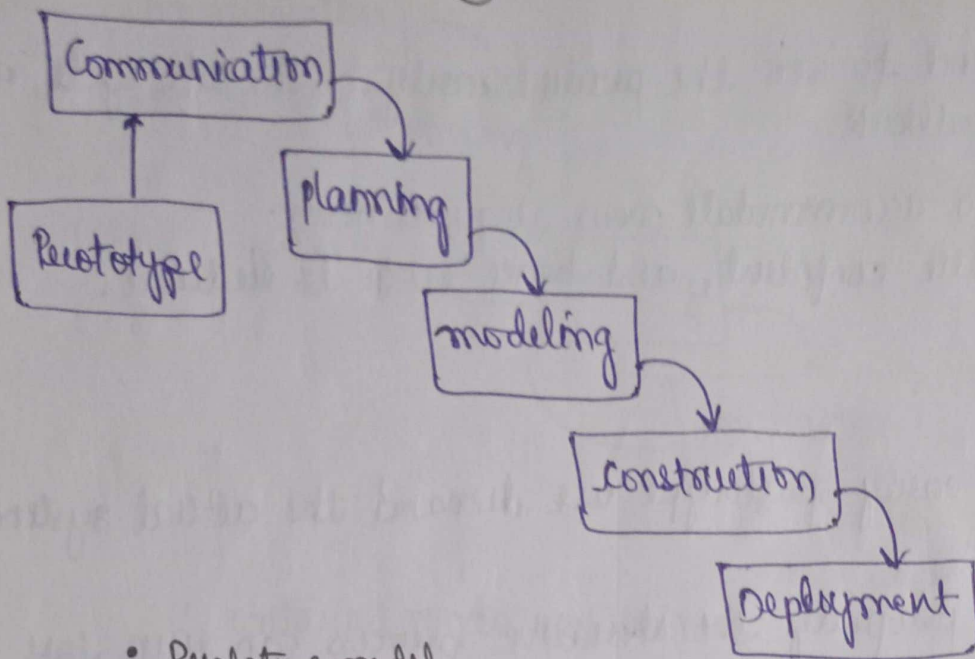
1. Customer get a chance to see the product early in the life cycle, and give important feedback.
2. There is a scope to accommodate new requirements.
3. Developer and more confident, and hence risk is reduced.

⑥ Disadvantage:-

1. After seeing the early prototype use demand the actual system soon.
2. If not managed properly the iterative process can run for a long time.
3. If user is not satisfied, he may lose its interest in the project.



OR

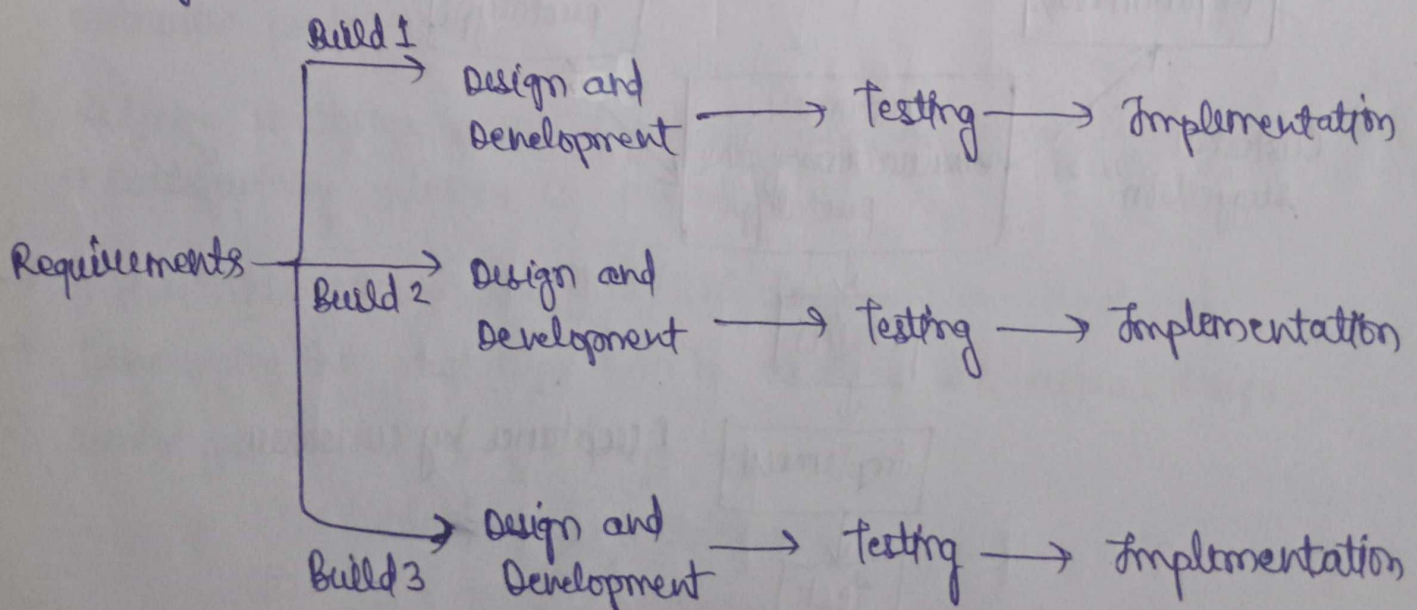


∴ Prototype model.

② Incremental process Model

② Incremental Model is a process of software development where requirements are broken down into multiple standalone modules of software development cycle.

→ Each iteration passes through the requirements, design, coding and testing phases.

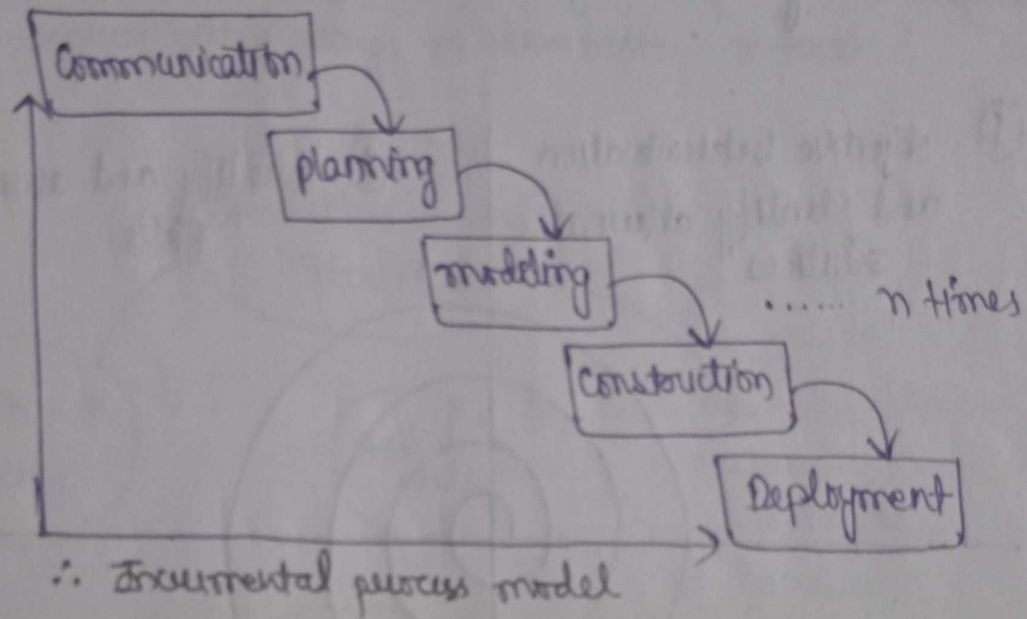


∴ Incremental process model

OR

① Advantages

1. Module by module working lead to less error.
2. For large product
3. Customer Interaction maximum.
4. Easy to understand and also maintain cost change.
5. Flexible to changes.



② Spiral Model

- ① The Spiral Model is a software development process that combines planning, risk analysis, development and testing in repeated cycles (called spirals).

→ with each cycle, the software is improved, and risk are addressed

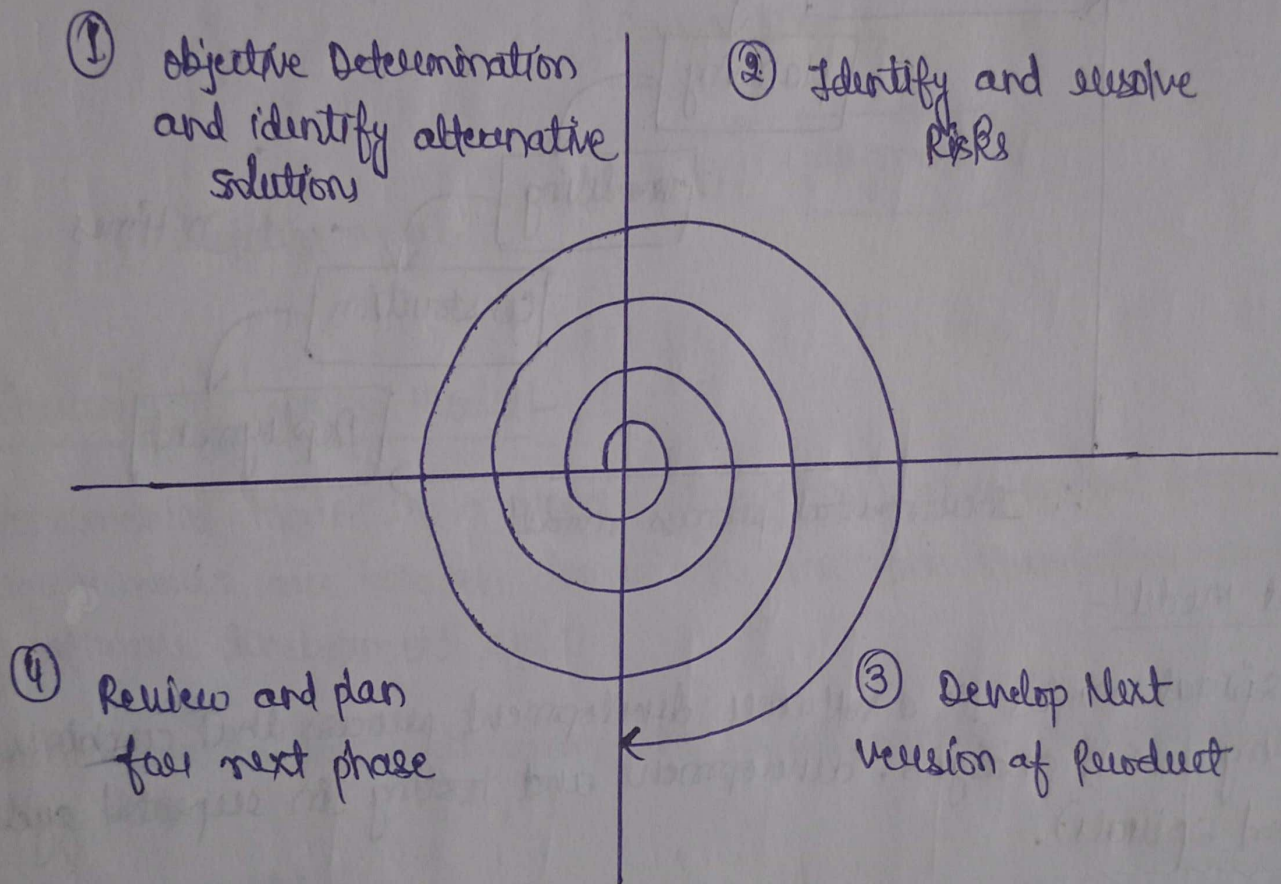
→ It's ideal for large projects where requirements may evolve over time.

③ Advantages:-

1. Risk management: Identifies and addresses risk in each iteration.
2. Flexibility:- Allows for changes and improvements during development
3. Regular feedback from customers helps refine the product.
4. Better for large projects.

⑥ Disadvantage:-

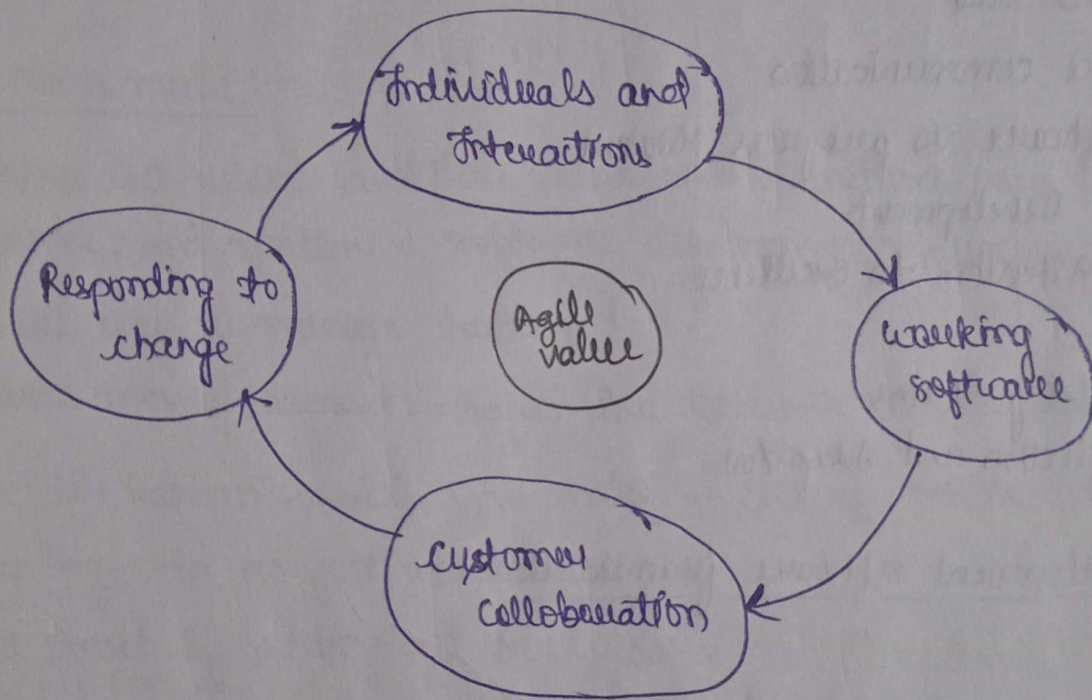
1. Not ideal for small projects.
2. Time-consuming.
3. Increase complexity.
4. Time-consuming.



→ x-axis clockwise through 360° represents one phase, which is divided into four sectors.

② Agile Software Development:-

- ① Agile is a flexible way of developing software where work is done in small parts, called "sprints", and changes are based on feedback.
- Agile is a set of 4 values and 12 principles.
- originated from the Manifesto for Agile software Development and aims to uncover better ways to developing software.



∴ Four values of Agile

③ Advantages

1. Frequent Delivery.
2. Face to Face communication with client.
3. changes can be done easily.
4. Time less.

④ Disadvantages

1. less documentation.

2. Maintenance Problem.

① 12 principles of Agile:-

1. Customer Satisfaction.
2. Welcome changing Requirements.
3. Frequent Delivery
4. Business and Developers collaboration.
5. Support and trust
6. Face-to-face communication
7. Working software as progress Measure.
8. Sustainable Development.
9. continuous Attention to Excellence.
10. Simplicity.
11. Self-organizing teams.
12. Regular Reflection and Adaption.

① Agile development software framework

1. SCRUM:-

① one of the most popular agile methodology.

→ Scrum is a lightweight, iterative and incremental framework.

⇒ Scrum is a popular Agile Framework used for managing and developing complex software projects. It emphasizes teamwork, iterative progress and continuous feedback.

→ It break down the development phases into stages called as cycles called "sprints".

① Advantages

1. Freedom & Adaption

2. High-quality, low-risk product
3. Reduce the development time up to 40%.
4. Reviewing the current sprint before moving to new one.

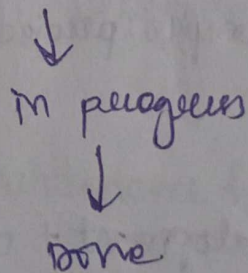
① Disadvantages:-

1. More efficient for small team size.
2. No changes in the sprint.

2. Kanban model:-

- ① Kanban is a visual workflow management method used to manage, improve, and optimize software development processes.

- Visual way to manage tasks.
- Tasks move through stages like todo → in progress → Done.
- In the Kanban model, you make a list of tasks and move them step-by-step as you work on them.
- You start by putting all tasks in a "To Do" column.

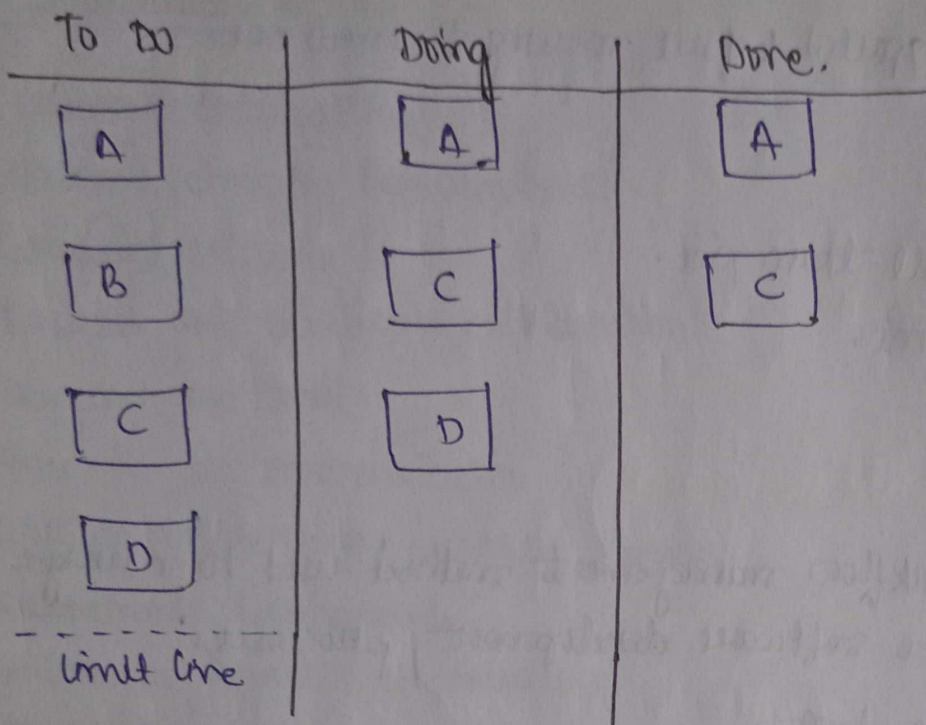


- This helps you stay focused, finish tasks faster, and see where work is stuck.

① Advantages

1. Flexibility (any change any time).
2. Improved efficiency.
3. Transparency (everyone can see work status).

Kanban Board



① Disadvantages:-

1. No fixed timelines
2. Scope creep risk.
3. Required discipline
4. Not ideal for complex projects.

① DevOps:-

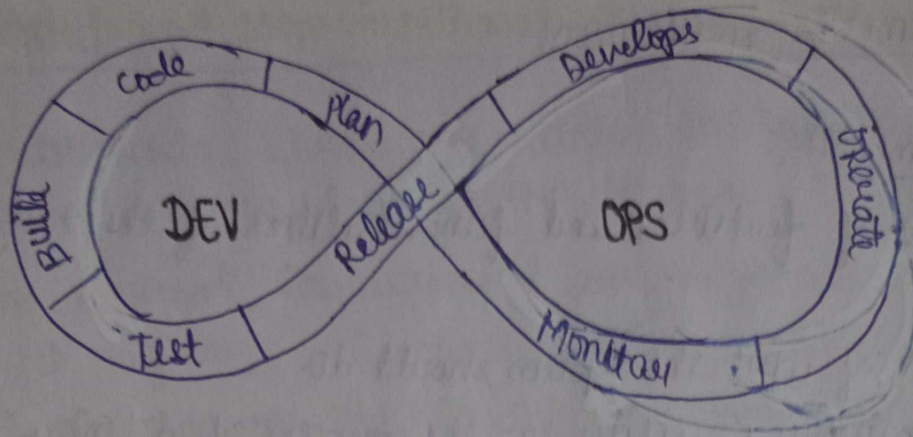
① DevOps = Development + operations.

① It means development and operations teams work together to build, test, and deliver software faster.

① It enhances collaboration between both.

① Advantages:-

- | | |
|------------------------------------|--|
| 1. Faster software delivery | 4. Improved collaboration between teams. |
| 2. Improved collaboration b/w them | 5. Better scalability. |
| 3. Continuous feedback. | 6. and reliability. |



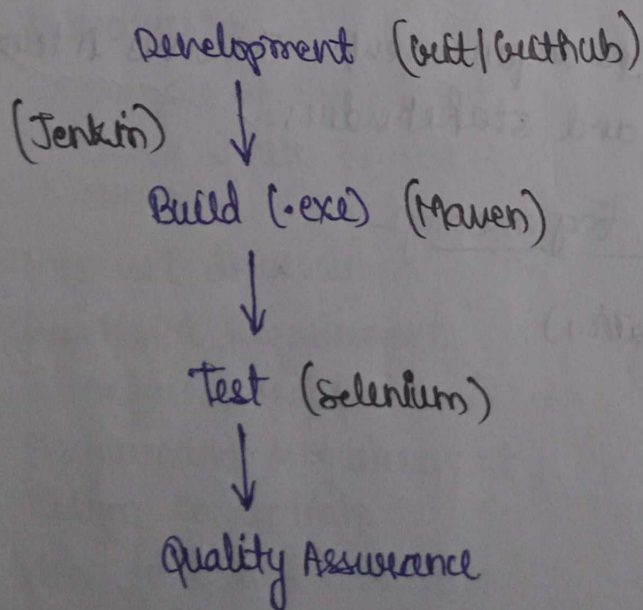
∴ Diagram

① Key DevOps principles

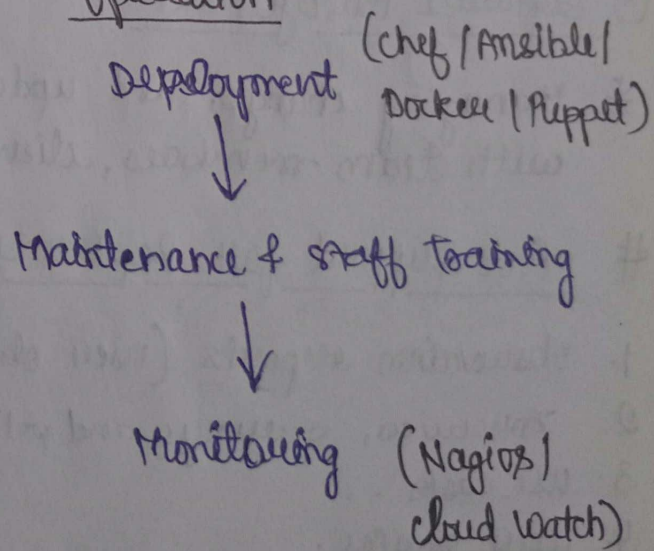
1. Collaboration.
2. Automation.
3. Continuous Integration & continuous Deployment (CI/CD)
4. Monitoring & Feedback.
5. Security.

② DevOps Lifecycle

Development



Operation



① Software Requirements Specification

Software requirements

① It is a description of features and functionalities of the target system.

→ It is a description of what the system should do.

⇒ Requirements engineering: refers to the process of defining, documenting and maintaining requirements in the engineering design process.

② It is a four step process, which includes -

1. Feasibility study.
2. Requirement gathering / Elicitation.
3. Software requirement specification.
4. Software Requirement Validation.

③ Software Management

* Managing changes and update to requirements. Involves interaction with team members, clients and stakeholders.

Tool support for Requirements Engineering

1. Observation reports (user observation).
2. Interview, surveys and polls.
3. Use cases.
4. User stories.
5. Requirement workshop.
6. Mind-mapping.
7. Role-play.
8. Prototype.

⑥ Functional and Non-Functional Requirements:-

- ① Requirements, which are related to functional / working aspect of software fall into the category.
- ② Non-Functional Requirements are expected characteristics of target software. (Security, storage, configuration, performance, cost, interoperability, flexibility, disaster recovery, etc.)

⑥ Software requirement Specification (SRS):-

- ⑥ SRS is a description of a software system to be developed.
- It lays out functional and non-functional requirements of the software to be developed.
- It may include a set of use case that describe user interactions that the software must provide to the user for perfect interaction.

SRS structure:-

1. Introduction
 - purpose of this project.
 - Scope of the project.
 - Overview.
2. General description.
3. Functional requirement.
4. Interface requirement.
5. Performance requirement.
6. Design constraints.
7. Non-functional attributes.
8. Schedule and budget.
9. Bibliography.

① User Requirement Specification

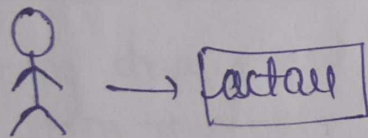
1. Easy & simple to operate.
2. Quick response.
3. effectively handling operational issues.
4. Customer support

→ The user requirement document or user requirement specification is a document usually used in software engineering that specifies what the user expects the software to be able to do.

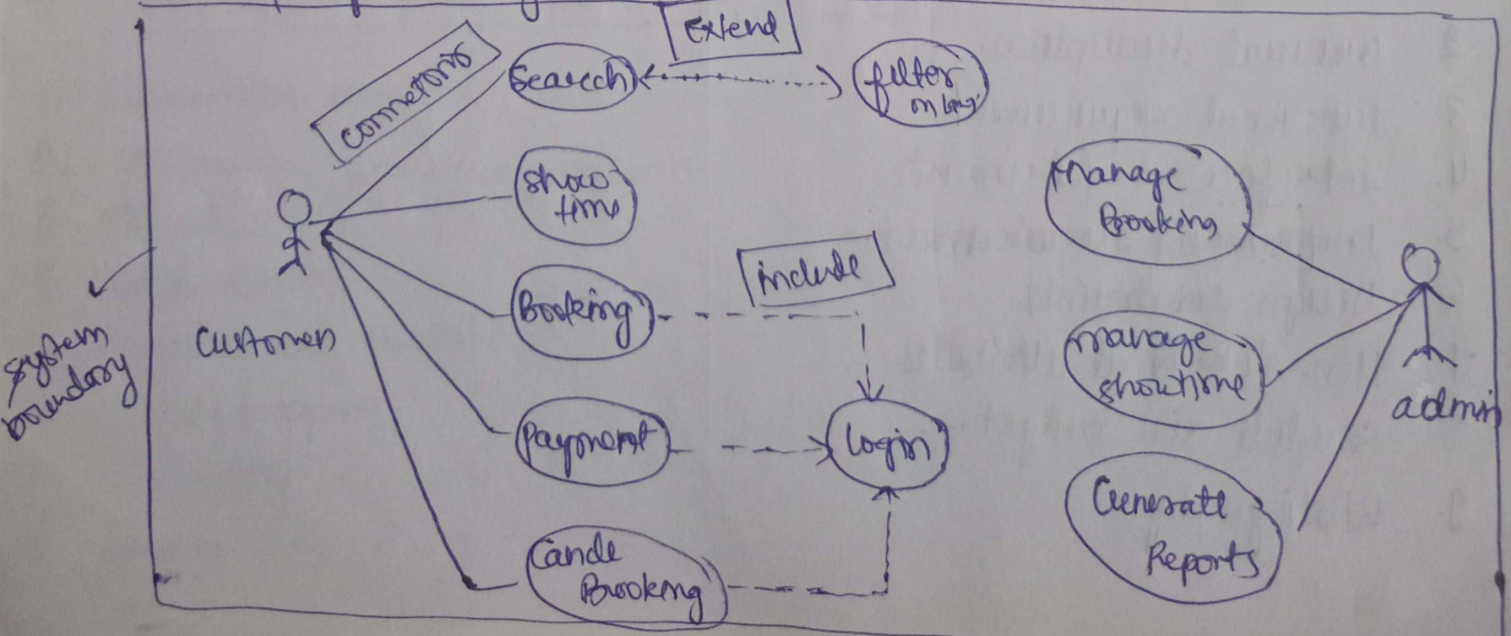
→ It is a contractual agreement.

② Use Case Diagram

- Use case diagram model the behaviour of the system.
- Used to illustrate the functional requirement of the system and its interaction with external agents (actors).
- A use case diagram gives us a high level view of the system without going into implementation details.

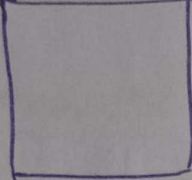


Ex: Example of Book my show website.



(i)  - actor

(ii)  - use case

(iii)  - system boundary.

① Software project planning

① project planning is one of the most important activities and is an engaging effect through out the life of the project.

- software project management is an art and science of planning and leading software projects.
- various factors make this job more complex (eg. changeability, complexity, uniqueness, possibility of multiple solutions etc.)

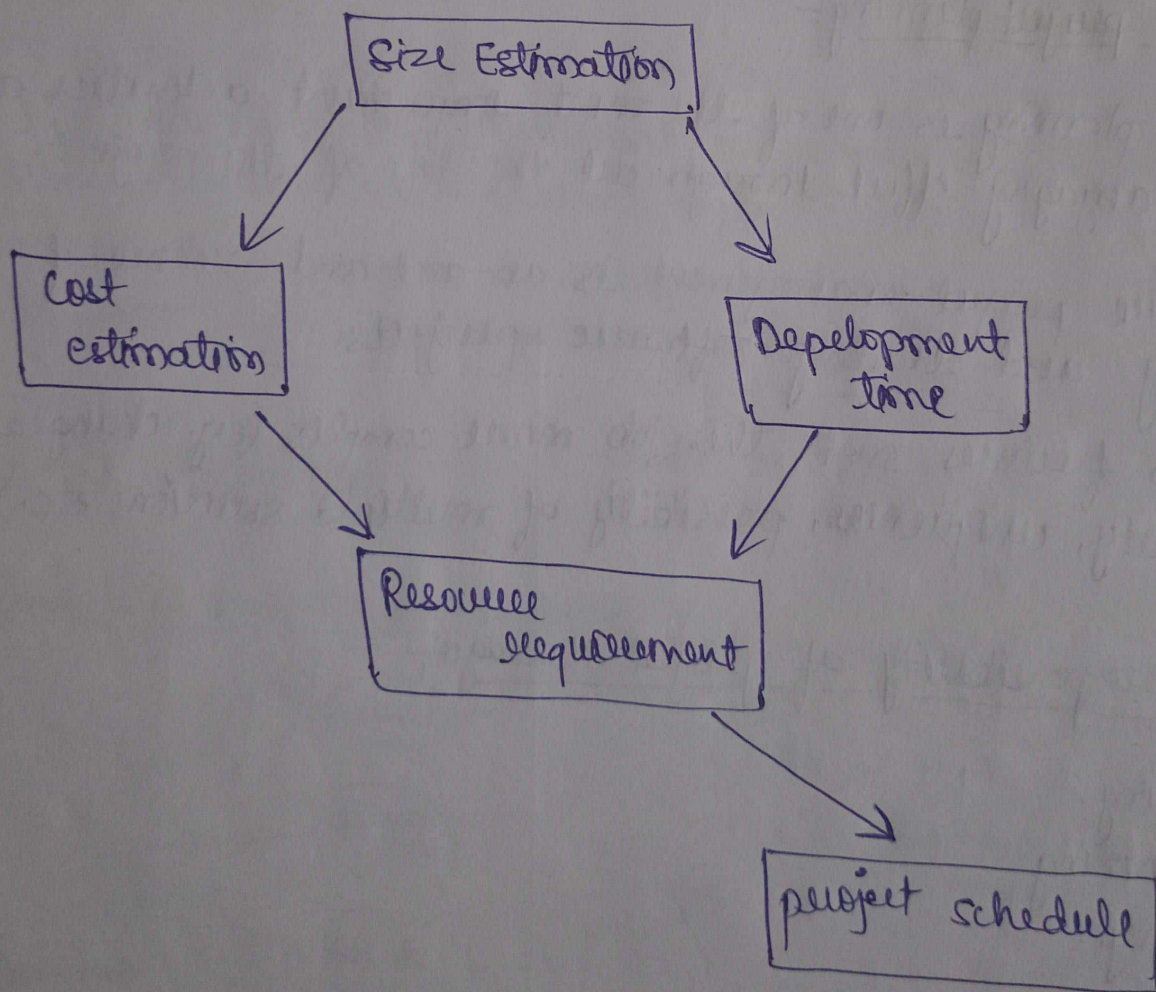
Job responsibility of project Manager:-

1. Planning.
2. Organizing.
3. Staffing.
4. Directing.
5. Monitoring.
6. Controlling.
7. Innovating.
8. Representing.

→ In order to conduct a successful software project we must understand

1. The risk to be increased.
2. The resource required.
3. Scope of work to be done.
4. The cost to be expended.
5. The schedule to be followed.

Various planning activities :-



① Software Metrics:-

Function point Analysis (FPA):-

- ① Function point Analysis (FPA):- Is a standardized method used to estimate the size, complexity, and functionality of software projects.
- ① Unadjusted Function points (UFP):- Calculate UFP by multiplying the number of each type of component by its corresponding weight and summing them up.
- ① Value Adjustment Factor (VAF):- A factor that adjusts the UFP based on general system characteristics (GISCs) which include considerations like performance, usability, and complexity.

$$FP = UFP \times (0.65 + 0.01 \times (\text{Sum of GISCs})) \quad (\text{Formula})$$

① List of 5 unadjusted Function Points:-

1. External Inputs (EI).
2. External outputs (EO).
3. External Inquiries (EQ).
4. Internal Logical Files (ILF).
5. External Interface Files (EIF).

They have three different complexity.

- └ Low.
- └ Average.
- └ High.

Component	Low complexity	Average complexity	High complexity
EI	3	4	6
EO	4	5	7
EQ	3	4	6
ILF	7	10	15
EIF	5	7	10

⇒ VAF depends on 14 factors.

⇒ 0 to 5

Ques. Consider a project with the following functional units:

Numbers of user inputs = 50

Numbers of user outputs = 40

Numbers of user enquiries = 35

Number of user files = 06

Numbers of external interfaces = 04

Assume all complexity adjustment factors are average. Compute the function points for the project.

$$\text{SOL} = \text{VFP} \times (0.65 + 0.01 \times (\text{sum of } \text{nsfs}))$$

$$\Rightarrow \text{VFP} = 50 \times 4 + 40 \times 5 + 35 \times 4 + 06 \times 10 + 07 \times 04$$

$$\Rightarrow \text{VFP} = 628$$

$$\Rightarrow \text{sum of } \text{nsfs} = 14 \times 3$$

$$\begin{aligned}
 &\Rightarrow 42 \\
 &\Rightarrow 628 \times (0.65 + 0.01 \times 42) \\
 &\Rightarrow 628 \times 1.07 \\
 &\Rightarrow \boxed{672} \text{ Ans.}
 \end{aligned}$$

Ques. An application has the following:-

→ 10 low external inputs, 12 high external outputs, 20 low internal logical files, 15 high external interface files, 12 average external inquiries, and a value of complexity adjustment factor of 1.10.

What are the unadjusted (VFP) and adjusted (CAF) function point counts?

Soln. $FP = VFP \times (0.65 + 0.01 \times (\text{sum of } VSCs))$

$$\Rightarrow VFP = (10 \times 3) + (12 \times 7) + (20 \times 7) + (15 \times 10) + (12 \times 4)$$

$$\Rightarrow VFP = 452$$

$$\Rightarrow FP = VFP \times CAF$$

$$\Rightarrow CAF = 1.10 \text{ (}\therefore \text{ Given)}$$

$$\Rightarrow FP = 452 \times 1.10$$

$$\Rightarrow \boxed{FP = 497.2} \text{ Ans.}$$

Ques Consider a software project with the following information domain characteristics for the calculation of function point metric.

$$EI \rightarrow 30$$

$$EO \rightarrow 60$$

$$EQ \rightarrow 23$$

$$ILF \rightarrow 08$$

$$EIF \rightarrow 02$$

It is given that the complexity weighting factors for I, O, E, F and N are 4, 5, 4, 10 and 7, respectively. metric is.

Soln $\Rightarrow 30 \times 4 = 120$

$$\Rightarrow 60 \times 5 = 300$$

$$\Rightarrow 23 \times 4 = 92$$

$$\Rightarrow 10 \times 08 = 80$$

$$\Rightarrow 02 \times 7 = 14$$

$$\boxed{\text{Total} = 606} = \text{UFP}$$

$$\Rightarrow \text{FP} = \text{UFP} (0.65 + 0.01 \times (\text{sum of GSCs}))$$

$$\Rightarrow \text{GSC}_p = 4 \times 0 + 4 \times 3 + 6 \times 4$$

$$\Rightarrow \text{GSC}_p = 0 + 12 + 24$$

$$\Rightarrow \text{GSC}_p = 36$$

$$\Rightarrow \text{FP} = 606 (0.65 + 0.01 \times 36)$$

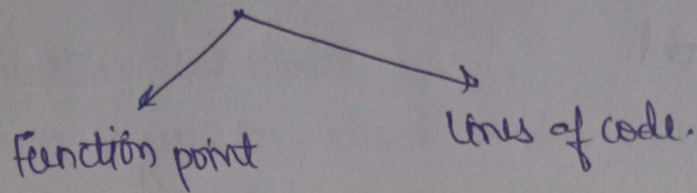
$$\Rightarrow \text{FP} = 606 \times (0.65 + 0.36)$$

$$\Rightarrow \text{FP} = 606 \times 1.01$$

$$\Rightarrow \boxed{\text{FP} = 612}$$

① Size

→ To estimate the size of project, for this we have two methods



① Function point. Measure the functionality delivered to the user based on user requirements and design

- Focus on user functionality and system behaviour.
- Based on user inputs, outputs, inquiries, files and interfaces.
- Independent of programming language.

① Lines of code:- Measure the physical lines of source code written in a programming language.

- Focus on the amount of code written.
- Dependent on the programming language, platform and technology.
- Based on counting the lines of code written.

What is not a line of code?

→ Blank line (improve readability of code).

→ comment line. (help in understand code).

Feature of lines of code

1. Change tracking

2. Limited representing of complexity.
3. Ease of computation.
4. Easy of understand.

Advantages

1. Effort Estimation. (Quick).
2. Comparative Analysis.
3. Simplicity.
4. Easy comparison.

Disadvantages

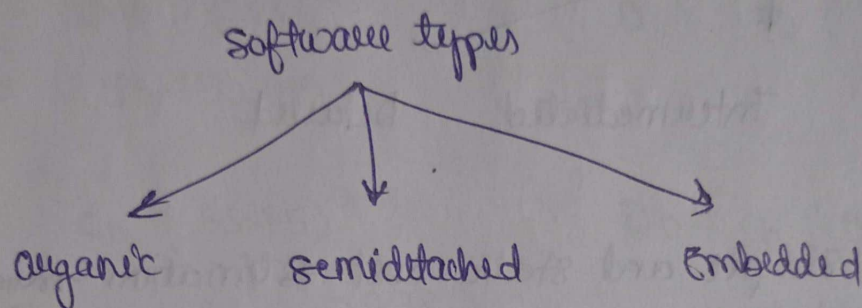
1. Language dependency.
2. Ignores code quality.
3. Maintenance challenges.
4. Difficult for modern development.

① Project Estimation Methods - COCOMO Model:-

COCOMO:-

→ Construction cost Model

It is a regression-based model developed by Barry Boehm, he postulated that there are essentially three important classes of the software.



- ① Organic - Relatively small group work in a familiar environment to develop well understood application program, here little innovation is required, constraints and deadlines are few and the development environment is stable.

Ex:-

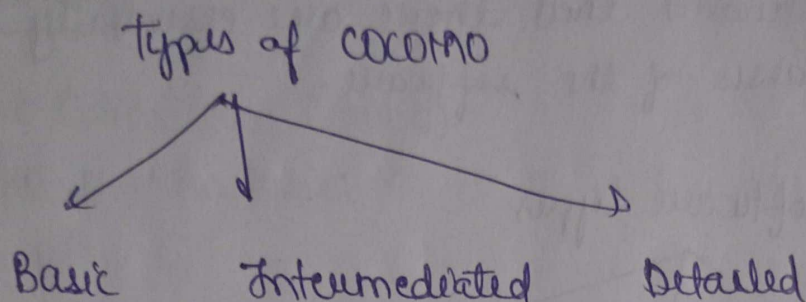
- ② Semidetached:- Project teams consist of a mixture of experienced and inexperienced staff. It is of medium size, some innovation is required, constraints and deadlines are moderate and the development environment is somewhat fluid.

Ex:-

- ③ Embedded:- The software is strongly coupled to complex hardware.

Such as air traffic control, ATM's air weapons system.

- The project team is large.
- Innovation is required, constraints and deadlines are tight
- development environment consist of many complex interfaces.



⑥ Basic

A simple and static cost estimation model that calculates project effort and duration based on the size of the software, measured in thousand of lines of code (KLOC).

$$\Rightarrow DE = a_b (KLOC)^{b_b} \quad (\text{development efforts})$$

$$\Rightarrow DP = c_b (DE)^{d_b} \quad (\text{development duration})$$

$$\Rightarrow TS = DE / DP \quad (\text{team size})$$

⑥ Table:-

Soft. Project	a_b	b_b	c_b	d_b
Organic	2.4	1.05	2.5	0.38
Semidetached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Ques:

$$KLOC = 20$$

Software type = organic

Find :-

$$\text{Effort} = ?$$

$$\text{Duration} = ?$$

$$\text{Team size} = ?$$

Soln :-

$$DE = 2.4 \times (20)^{1.05}$$

$$(\because DE = a_b (KLOC)^{b_b})$$

$$= 55.75 \text{ PM}$$

$$\Rightarrow DO = c_b (55.75)^{0.38}$$

$$(\because DO = c_b (DE)^{d_b})$$

$$= 2.5 (55.75)^{0.38}$$

$$= 11.52 \text{ M}$$

$$\Rightarrow \text{Team size} = \frac{DE}{DO}$$

$$= \frac{55.75}{11.52}$$

$$= 5$$

Staffing :-

- ① Staffing in software engineering involves hiring, managing, and retaining skilled professional to build and maintain software effectively.

Understanding project needs

- identifying the required roles (developers, testers, project managers)
- Defining skill sets needed (Java, python, Devops, AI/ML)
- Estimating the numbers of team members based on project scope and deadlines.

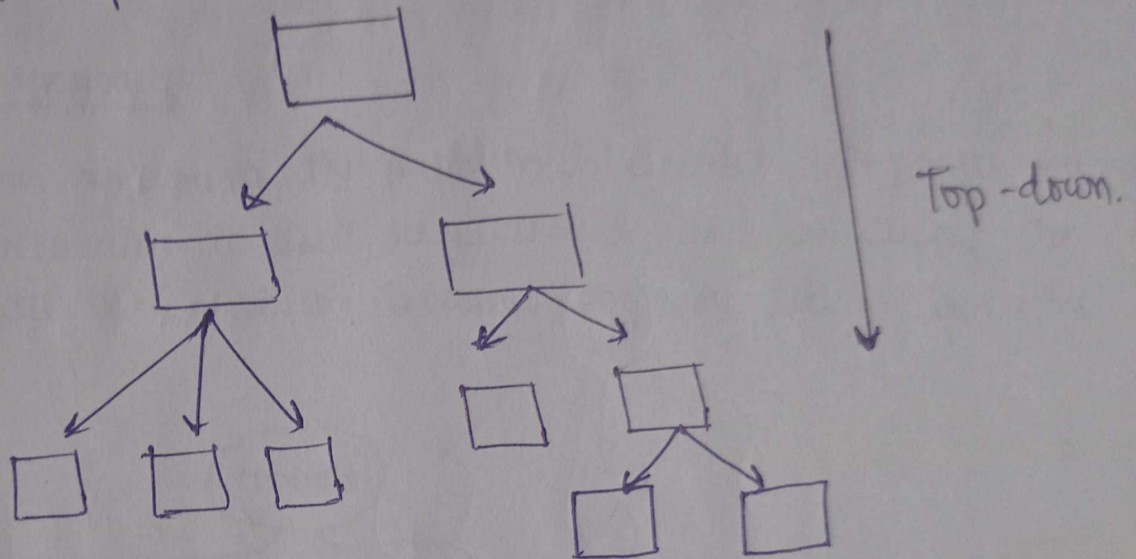
→ Staffing process

1. planning.
2. Recruitment.
3. selection
4. Hiring & onboarding.
5. Assignment.
6. Performance Monitoring
7. Retention.

① Function-oriented Design

① System is designed from a functional view point.

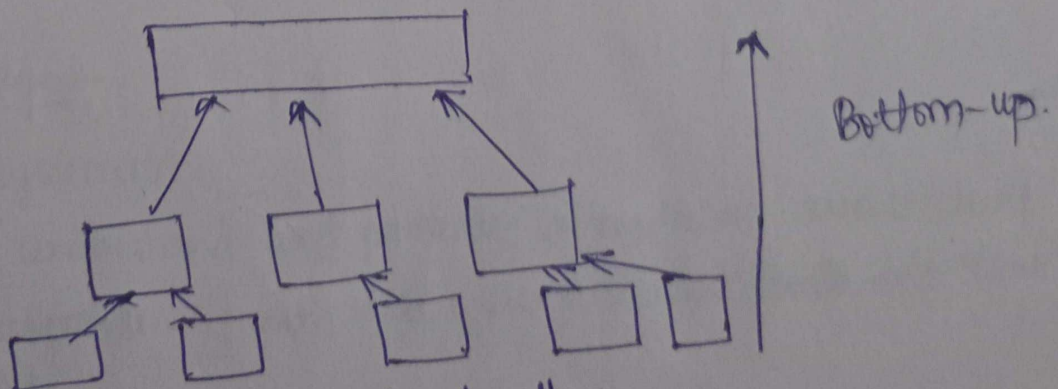
- Top-down decomposition.
- Divide & conquer approach.
- DFD is used.
- Divide and conquer rule.



② Object oriented Design

① System is viewed as a collection of objects (i.e., entities)

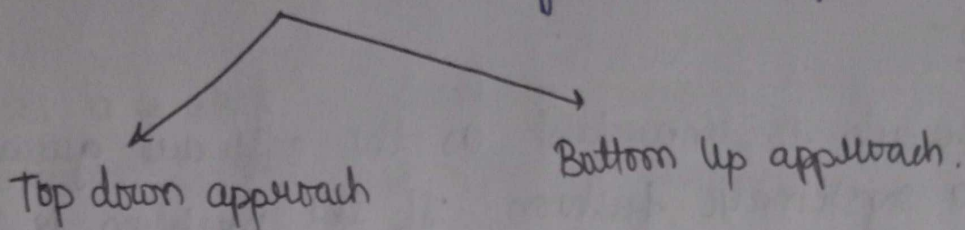
- Bottom-up approach.
- UML is used.



- Focuses on data and methods together.

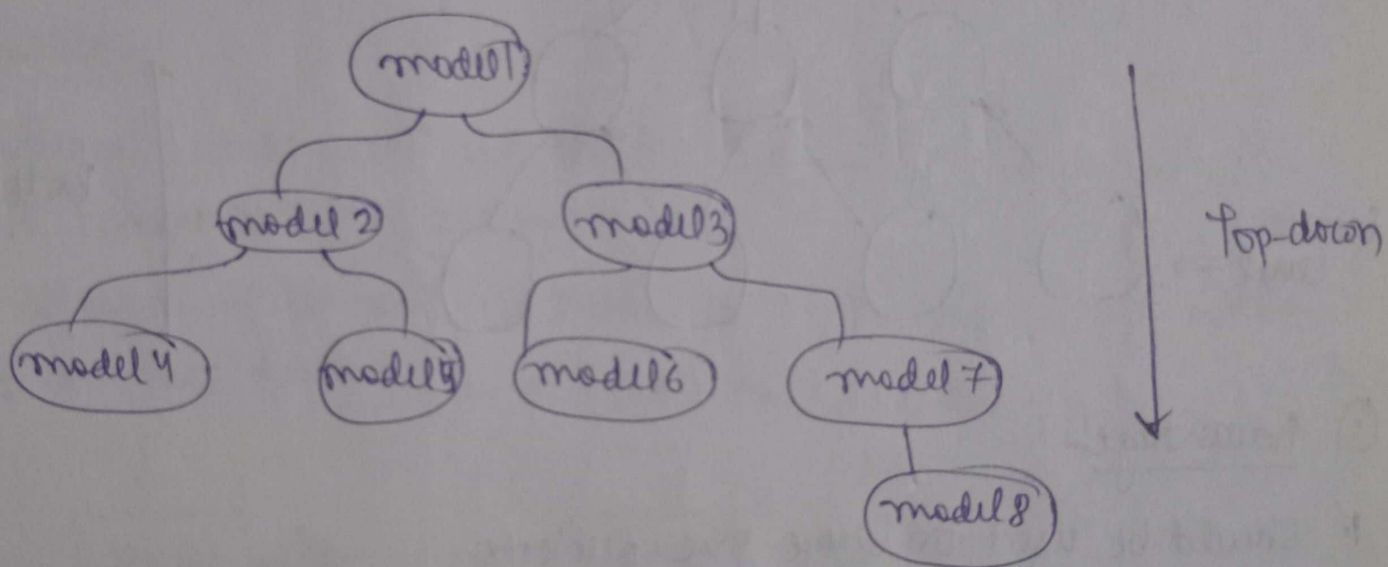
① Design Approach:-

② There are two popular approach using which design is done.



(1) Top down approach:-

② In top down approach, the problem is divided into small no. of programs according to user requirements and we repeat the process until the problem becomes so easy till we can solve it directly.



② Advantage:-

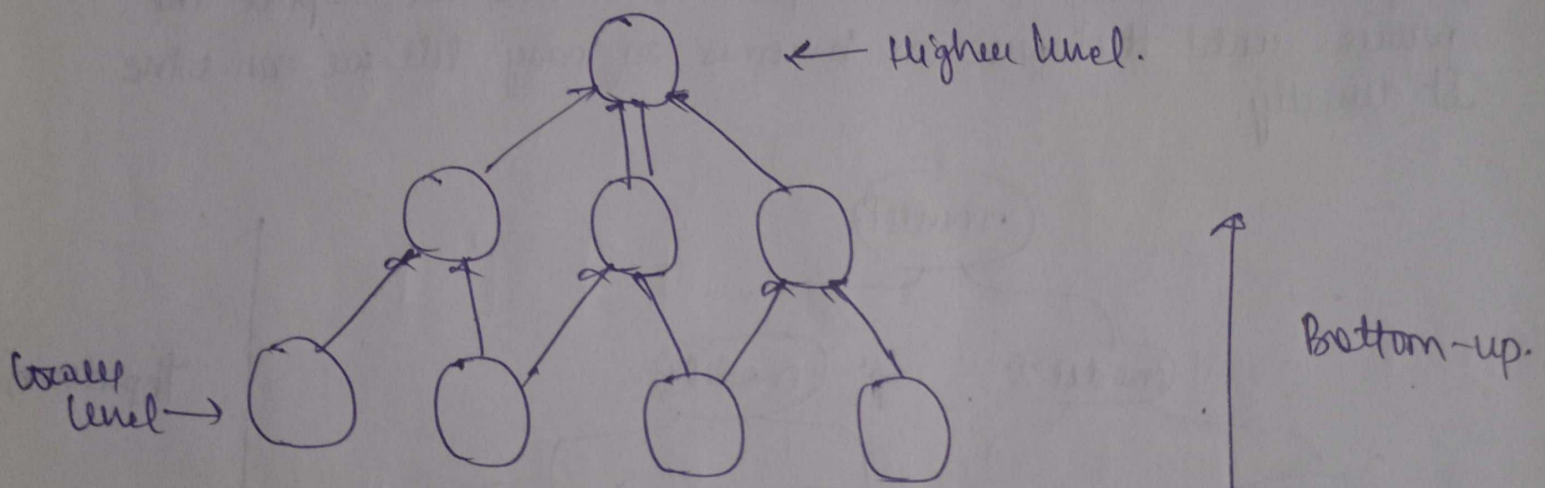
1. More systematic.
2. Easy to understand and provide a modular architecture.
3. In correspondence with user requirement cost and time prediction is possible.
4. Should be used with small and medium size products.
5. Used mostly.

③ Disadvantage:-

⑤ When a problem is very complex and very large then we can not understand the entire problem as a whole.

⑥ Bottom-up:-

⑥ This approach is important as the software grows in complexity in a systematic fashion. If the problem is very difficult to understand instead of understanding the entire problem, we must solve some sub-problem and then should keep repeating the process until the entire problem is solved.



⑦ Advantages

1. Should be used on large size projects.
2. Easy to use by designers as we work in incremental fashion.

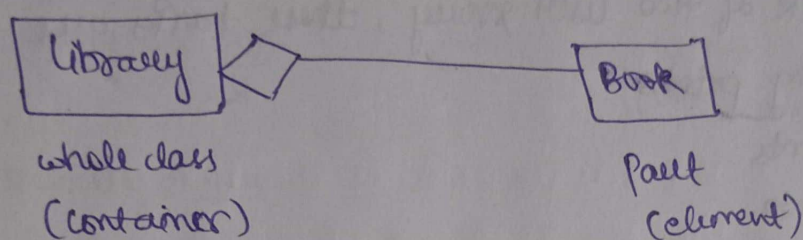
⑧ Disadvantage:-

- ⑧ It has complex architecture very difficult to understand and manage.

⑥ Aggregation:-

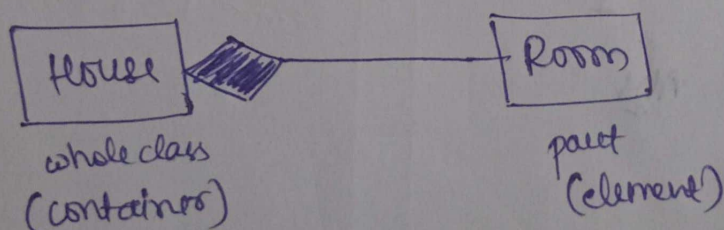
- A 'whole-part' relationship where the part can exist independently of the whole.
- It signifies a weaker relationship compared to composition.
- If container destroy but element cannot

Ex:-



⑦ Composition:-

- A strong "whole-part" relationship where the part cannot exist independently of the whole.
- The lifecycle of the part is tied to the lifecycle of the whole.
- If container destroy, so element also destroy.



① Velocity Numericals

→ Formula :-
$$\frac{\text{Total story points completed in a sprint}}{\text{Number of sprints}}$$

Ques 1: In a specific sprint the cross function team decided to implement 7 different task of two user story, these tasks are estimated to have following points:-

Task 1; 4 points.

Task 2; 6 points.

Task 3; 10 points.

Task 4; 4 points.

Task 5; 1 points.

Task 6; 2 points.

Task 7; 6 points.

Soln.

$$\text{Velocity} = \frac{4+6+10+4+1+2+6}{1}$$

$$\Rightarrow \boxed{33} \text{ Ans}$$

Ques 2: In 4 sprints

//

//

//

// ?

Task 1; 8 points.

Task 2; 10 points.

Task 3; 7 points.

Task 4; 3 points.

task 5; 7 points.

task 6; 2 points.

task 7; 1 points.

sm.

$$\text{velocity} = \frac{8+10+7+3+7+2+1}{4}$$

$$\Rightarrow \text{velocity} = \frac{38}{4}$$

$$\Rightarrow \text{velocity} = 9.5$$

② Module-level concept:-

Modularity:-

- ① In modular architecture, we understand that a system is composed of well defined conceptually simple and independence units interacting through a well-defined interface.

Coupling and cohesion:-

→ Coupling and cohesion are two parameters on which we can understand the quality of modularity in the design of a software.

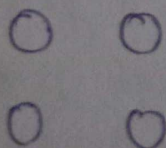
② Coupling:-

- ① The measure of interdependence of one module over another module

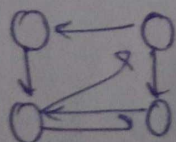
Types:-

Data coupling.
Stamp coupling.
Control coupling.
External coupling.
Common coupling.
Content coupling.

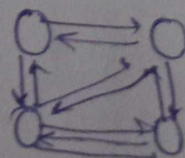
↑ Best (low)
↓ worst.



(i) uncoupled



(ii) loosely coupled



(iii) highly coupled.

① Data coupling:-

- ① These two modules communicate with each other only by passing data. It is most desired type of coupling. e.g. addition by using call by value is an example of data coupling.

② Stamp coupling:-

- ① When two modules communicate with each other by passing of data structure is called stamp coupling.
e.g. addition of two value but by call by reference.
→ we pass the address of the data.

③ Control coupling:-

- ① The module is set to be controlled coupled if they communicate using control information with each other.
→ e.g. dependences of two modules on each other because of flag.
(i) procedure-consumer.
(ii) token and signal.

④ External coupling:-

- ① In case of External coupling two modules on each other external to the logic of software (mostly because of hardware issue).
→ e.g. data bus, main memory etc.

⑤ Common couplings

- ① Two modules are set to be common coupled if they share some

global data.

→ eg. Synchronization issue between the process.

⑥ Content coupling:-

① When one module is a part or context of another module then it is called content coupling. It is worst type of coupling.

Eg. car and engine.

⑦ Cohesion:-

① We mean the measure of functions strength of the module.

→ Cohesion in software Engineering means how well the parts of a module work together to do one specific job.

Types:-

Function	↑ high low.
Sequence	
Communicational	
Procedural	
Temporal	
Logical	
Coincidental	

⑧ Coincidental cohesion:-

① As the name suggest is the worst kind of cohesion where the only relationship between the functions which are present in a module is random or coincidental.

② Logical cohesion:-

- ① All the elements of a module perform similar or slightly similar operations, for eg if mouse, prints, scanner functions are written in the same module.

③ Temporal cohesion:-

- ① Tasks are grouped because they need to happen at same time.

④ Procedural cohesion:-

- ① Tasks are grouped because they follow a specific order, but aren't directly related.

⑤ Communicated cohesion:-

- ① Tasks are related because they use the same data.

⑥ Sequential cohesion:-

- ① The tasks are related in sequence, where the output of one task is the input for the next.

Ex:- Reading data, then processing it, then saving it.

⑦ Functional cohesion:-

- ① Everything in the module does one specific task.

Ex:- A function that only calculates the sum of numbers.

① Design Notation:-

① Design Notation is a way to represent the structure and behaviour of a system using symbols, diagrams, and charts. It helps developers understand how different parts of the system work together.

Some common types of design notation:-

1. Flowchart
2. Data Flow diagrams.
3. UML Diagrams.

① Specification structure charts:-

① A structure chart is a diagram that breaks down a system into smaller parts (module) and shows how they are connected.

1. Top level-module
2. Submodules.
3. Connections.

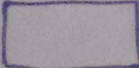
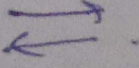
① Structured Design Methodology:-

Data Flow diagram:-

① A graphical tool, useful for communicating with users, managers, and other personnel.

→ Also called bubble chart.

Data Elements:-

1. Source / sinks (external entities) 
2. Data flow. 

3. processes ○

4. Data store / Data base =

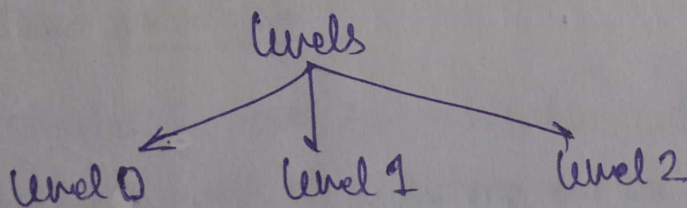
Rules of Data Flow

→ Data can flow from

1. External entity to process.
2. Process to external entity.
3. process to store and back
4. process to process

→ Data cannot flow from

1. External entity to external entity.
2. External entity to store.
3. store to store.
4. store to external entity.

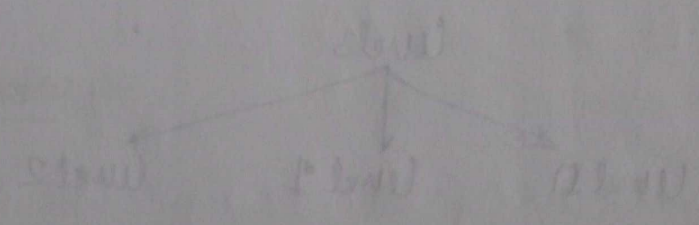
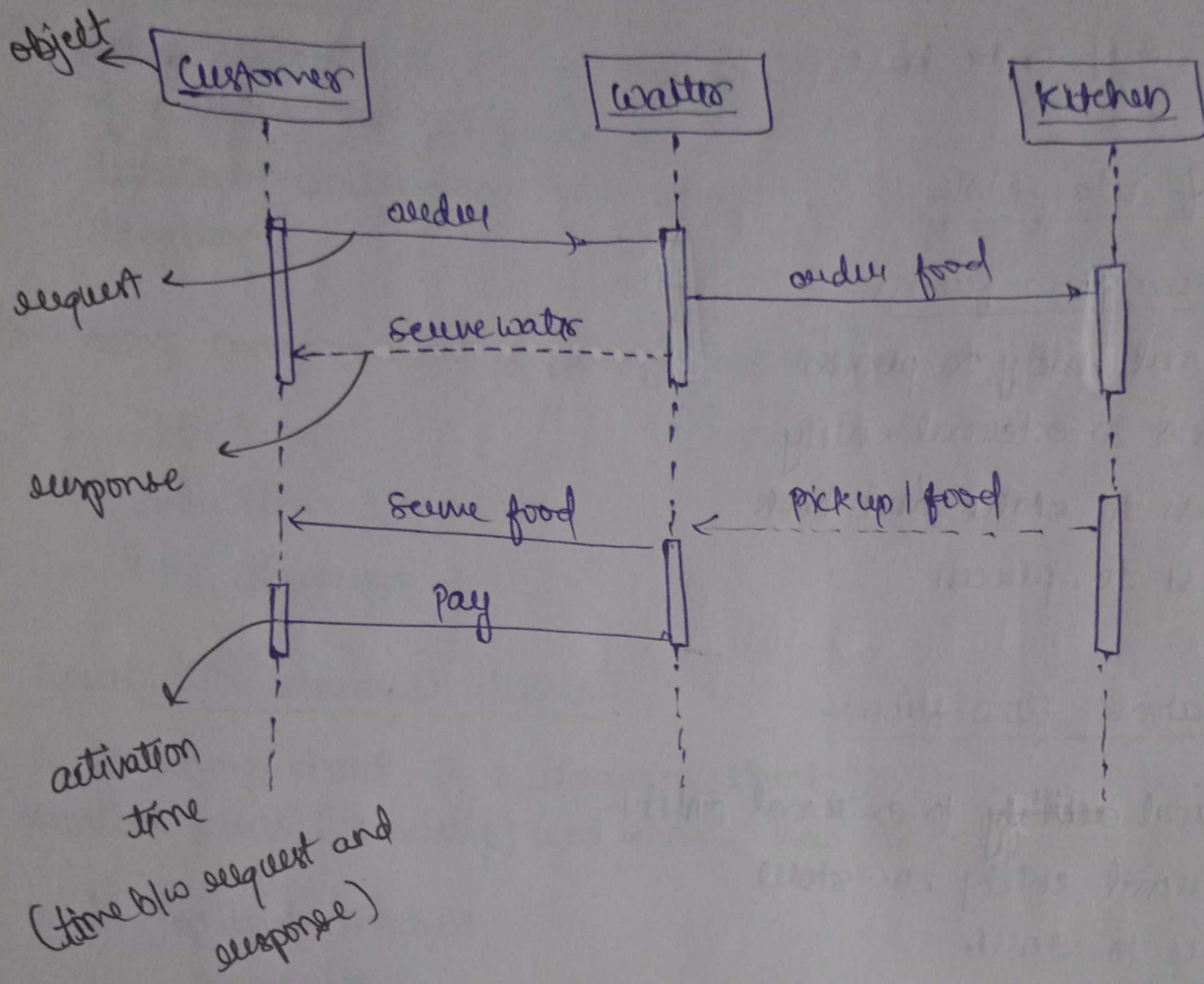


⑥ Sequence diagram

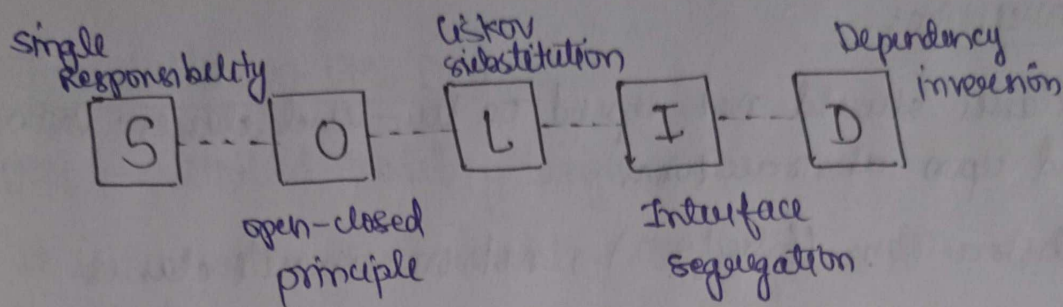
- ⑥ A sequence diagram in software engineering usually represents the interactions between different components or objects in a system over time.

Let's understand with the diagram.

UML Sequence diagram for restaurant



SOLID PRINCIPLES



① Introduced by Robert J. Martin.

② SOLID is a set of 5 rules that help make better, cleaner, and easier-to-maintain code, especially in oops.

1. Single Responsibility Principle

① A class should do one thing and therefore it should have only a single reason to change.

→ one job per class.

2. Open-closed principle

① A class should be open for extension but closed to modification.

→ Add features without touching old code.

3. Liskov substitution principle

① Subclass should be able to replace their parent classes without breaking the code.

→ Subclasses must work properly if used instead of the parent.

4. Interface segregation

① classes should not be forced to implement a function they do

not need.

→ No fat interfaces - split them into small ones.

5 Dependency Inversion

① High-level classes should not depend on low-level classes. Both should depend upon abstractions.

→ Depend on abstractions (interfaces), not on concrete classes.

② Advantages

1. Better code organization.

2. Easy to maintain.

3. Easy to test.

4. Promotes reusability.

③ Disadvantages

1. More complex code for small projects.

2. Too much boilerplate for small apps.

3. Risk of over-engineering.

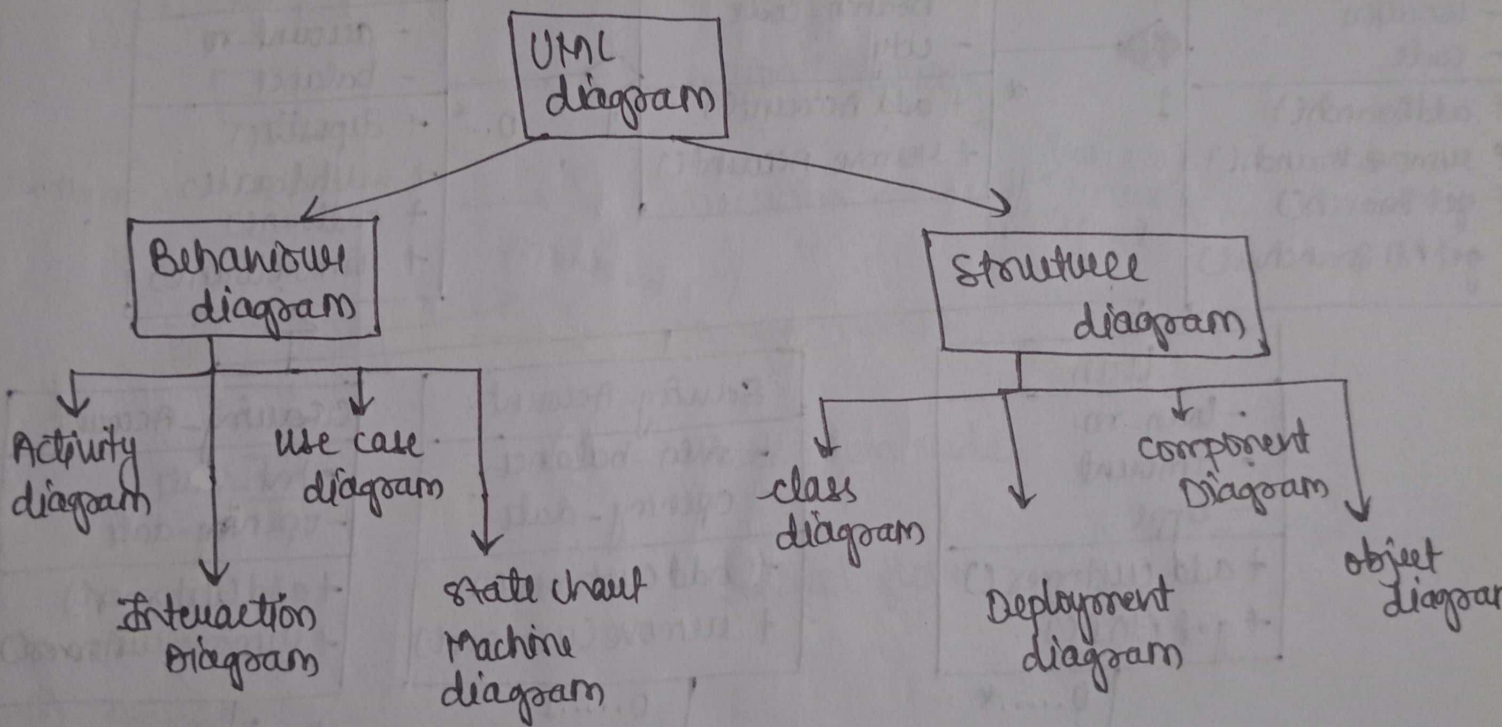
① OO Analysis and OO design?

OO concept-

Introduction to UML Design

② UML → Unified Modeling Language.

→ It is a standardized modeling language used to visualize, specify, construct, and document the design of software systems. It helps developers understand and design complex software architecture through diagrams.



③ Advantages

1. Enhance system understandability.
2. Promotes reusability and scalability.
3. Improves communication among developers.

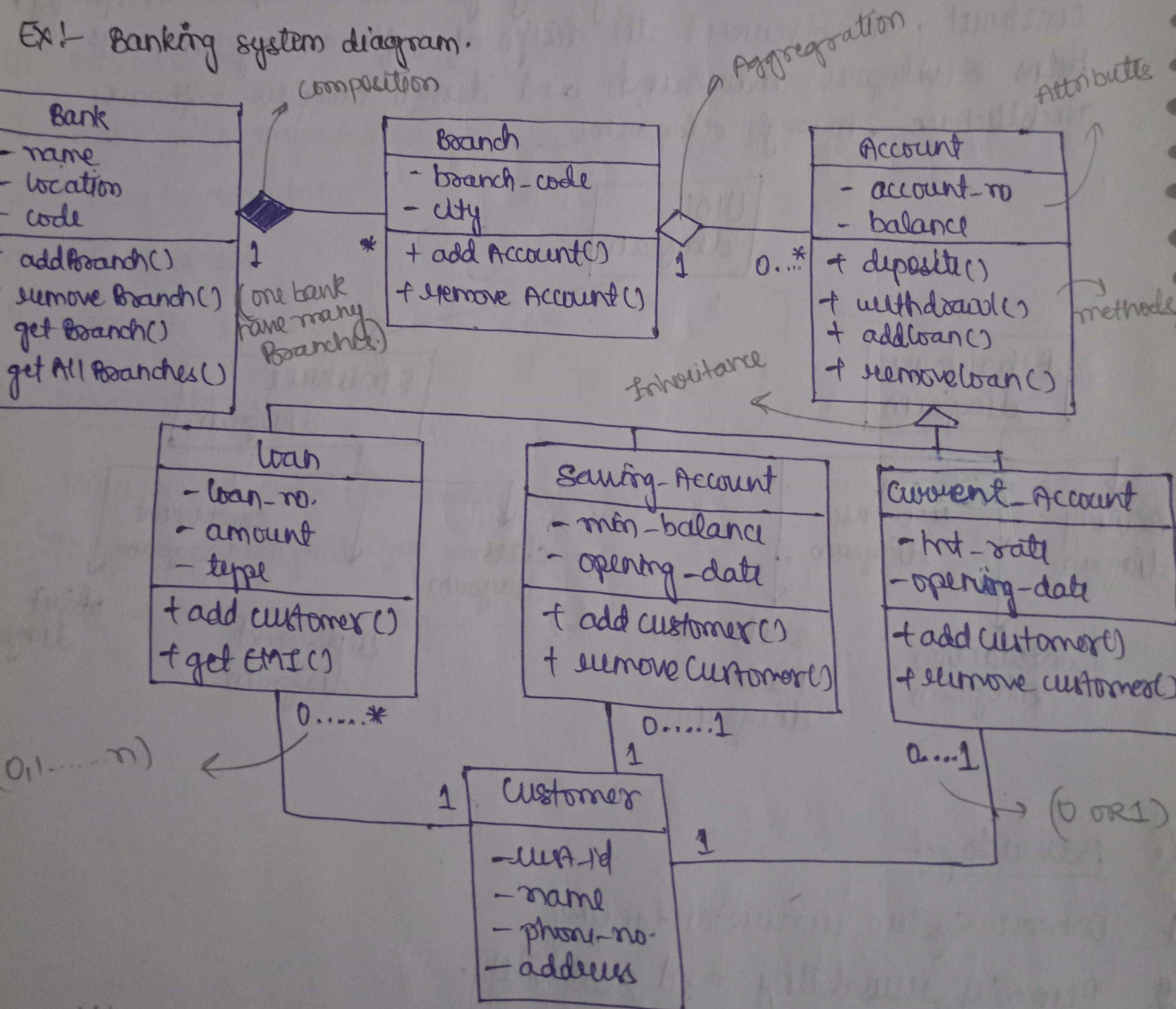
① class diagram

② class diagram are static diagram.

→ It represents the static view of an application.

A class diagram is a type of UML diagram that shows the static structure of a system. It represents the classes, their attributes, methods, and the relationship between them.

Ex:- Banking system diagram.



+ public
- private
protected
~ package

① Activity Diagram

② An Activity Diagram is a UML behaviour diagram that shows the flow of control or actions in a system just like a flowchart. It describes how the system behaves in response to different events.

③ Basic components of an Activity diagram

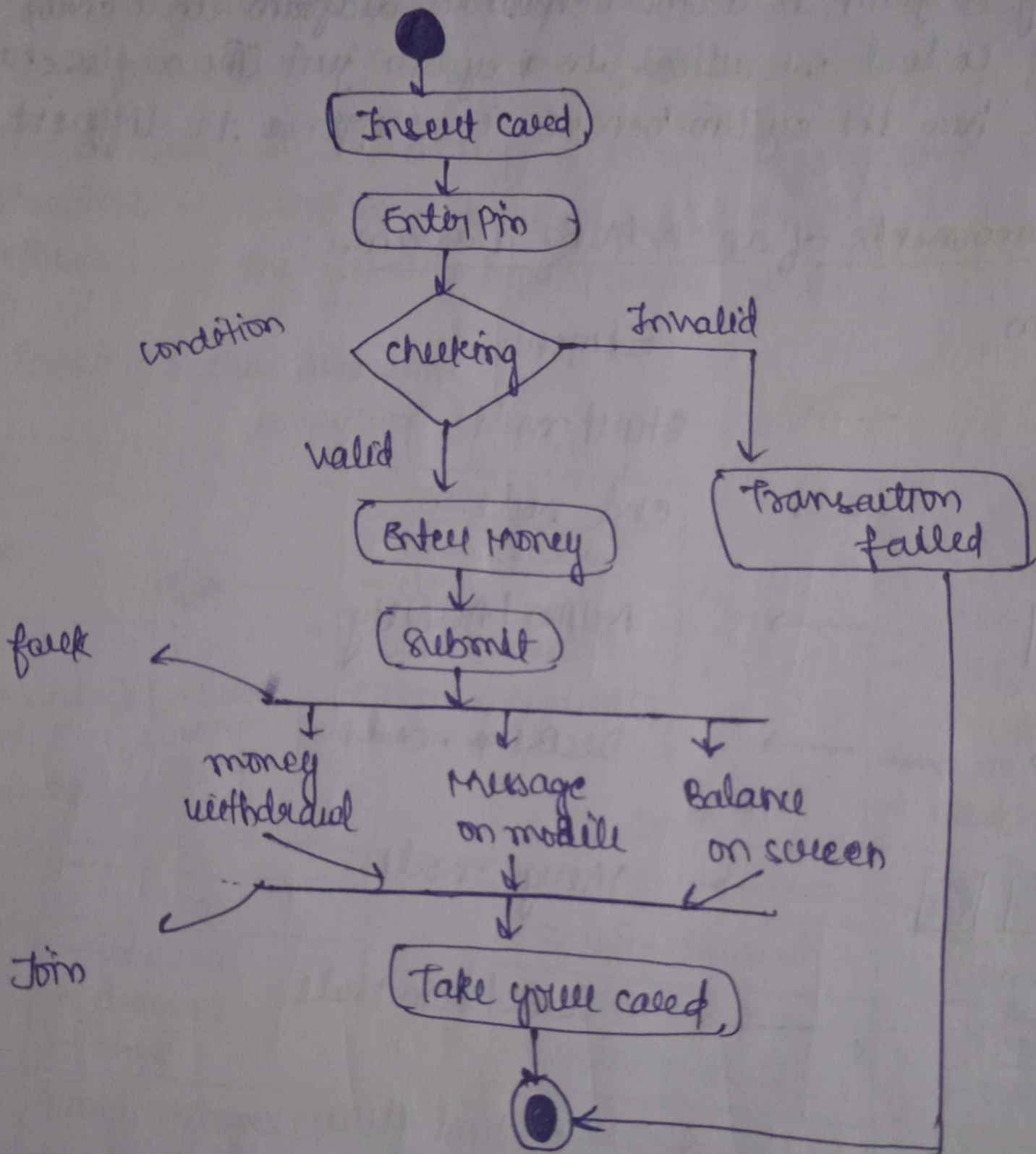
Symbol	Element
●	start node
⦿	end node.
□	Action / Activity.
◇	Decision node.
▢ ▢ ▢	Merge node.
≡	Fork / Join Node
→	control flow.

④ Advantages

1. Easy to understand.
2. Visualizes workflow clearly.
3. Helps in analyzing complex logic.
4. Helpful in software design.

Ex:-

ATM transaction:-



① State chart Diagram:-

① A state chart Diagram shows that how an object changes from one state to another based on events or actions.

② Key elements of a state chart diagram:-

Symbol	Elements
●	Initial state
⦿	Final state
□	state
→	transition
[]	Event / Trigger

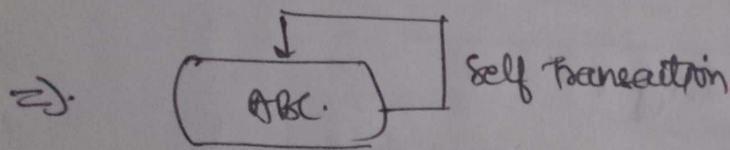
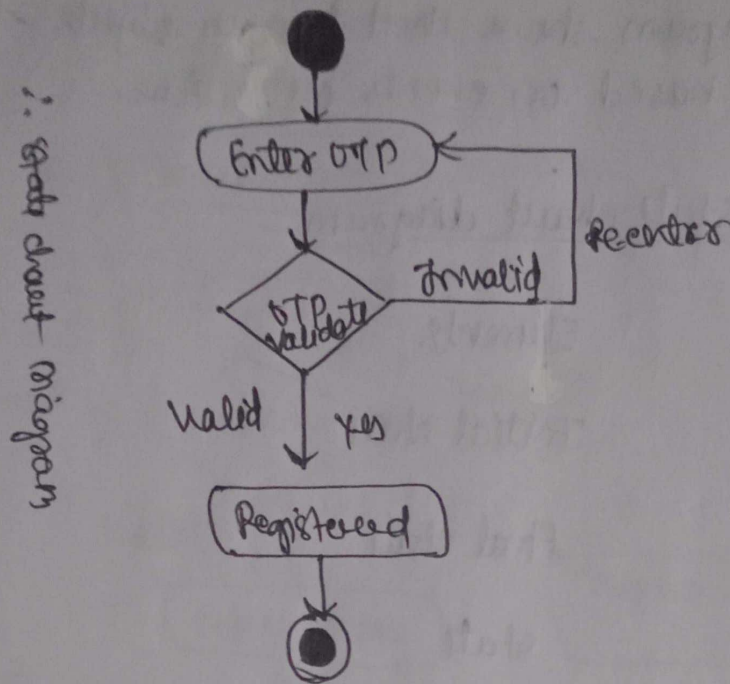
③ Advantages:-

1. shows object behaviour clearly.
2. Help in designing complex system.
3. makes logic easy to understand.
4. useful in testing and debugging.

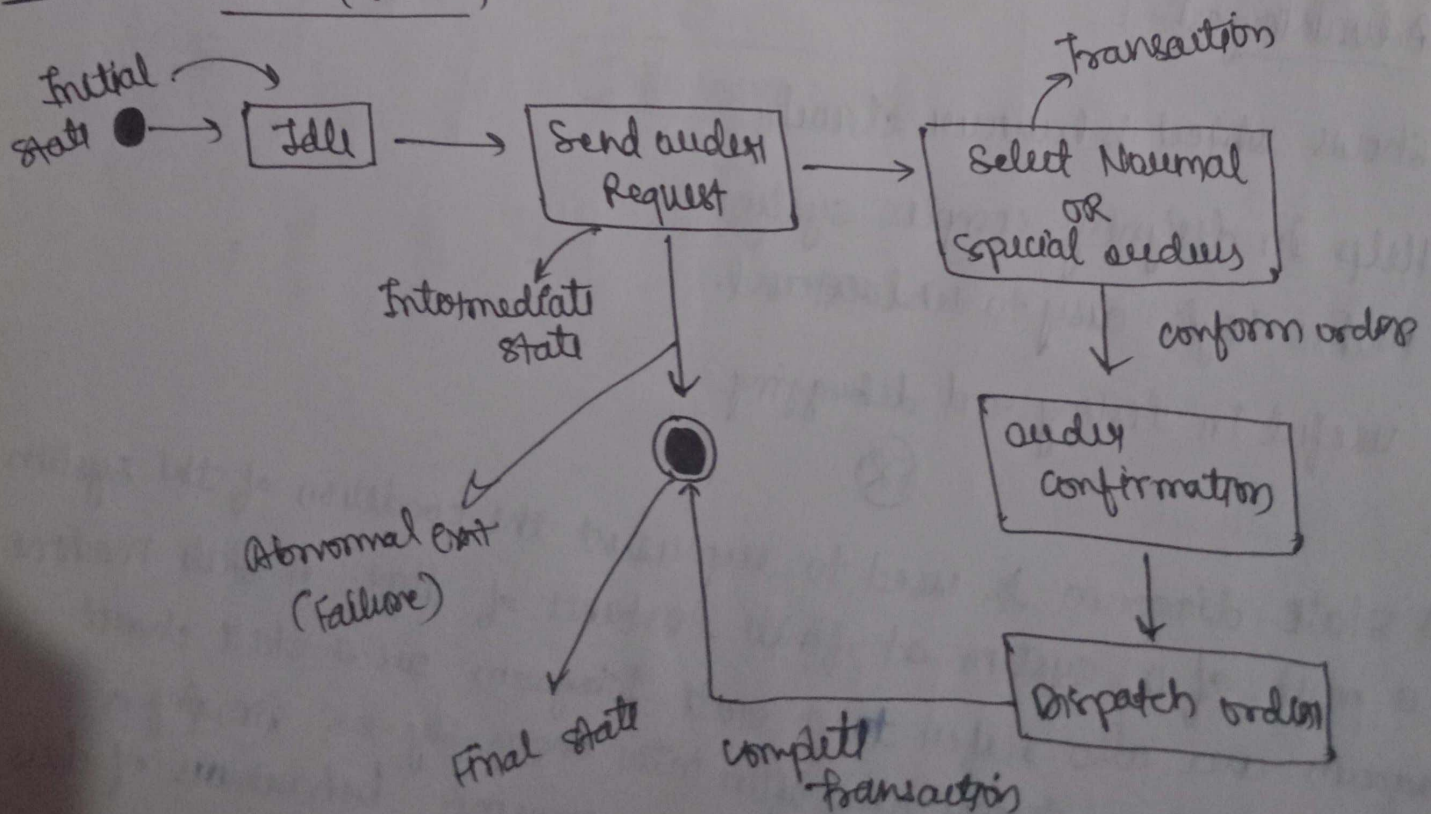
OR

④ A state diagram is used to represent the condition of the system or a part of a system at finite instance of time. A state machine Diagram or a state chart Diagram are also refer to a state Diagram or a state chart Diagram. These terms are often interchangably so, simply a machine diagram is used to model dynamic behaviour of class in response to time and changing external simulations.

Ex (i) student registration:-



Ex (ii) order (online):-



TESTING FUNDAMENTAL

① Testing Fundamentals

① Testing

② Because of human error, there will be a bug or fault in the code and if that bug and fault is executed, it becomes a failure.

③ Importance of testing

1. Detects bugs and errors early.
2. Enhance security
3. Improves user experience and reliability.

④ Test case Design

⑤ Test case Design refers to the process of creating a set of inputs, execution conditions, and expected results to validate the correctness and functionality of a software application.

⑥ What does a Test case include

1. Test case ID.
2. Test Description.
3. Test steps.
4. Input data.
5. Expected Result
6. Actual Result
7. Status.

⑦ How to design a good test case:-

- Think like a user.
- Cover all possible situations.
- Keep it simple and clear.

- Reuse test cases if possible
- Include edge cases

① Popular Test case design Techniques:-

1. Black Box Testing
2. White Box Testing.
3. Gray Box Testing.

① Black Box Testing strategies:-

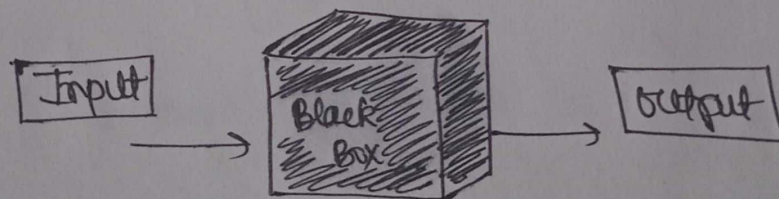
- ① Where we treat system as a whole, and check system according to user requirement (we are making the right product)
i.e. we check o/p for every i/p.

OR

- ① It is a method of software testing where you test the software from the outside, without knowing the internal code or logic

Ex:- Think it like a remote control. you press buttons (inputs) and see if the TV changes channels (outputs), you don't need to know how the remote works inside.

- It is called black box because you can't see inside.



- It is performed by test engineers.

① Advantages:-

1. No need to know the code.
2. Helps find missing functionalities.
3. Simulates real user behaviour.

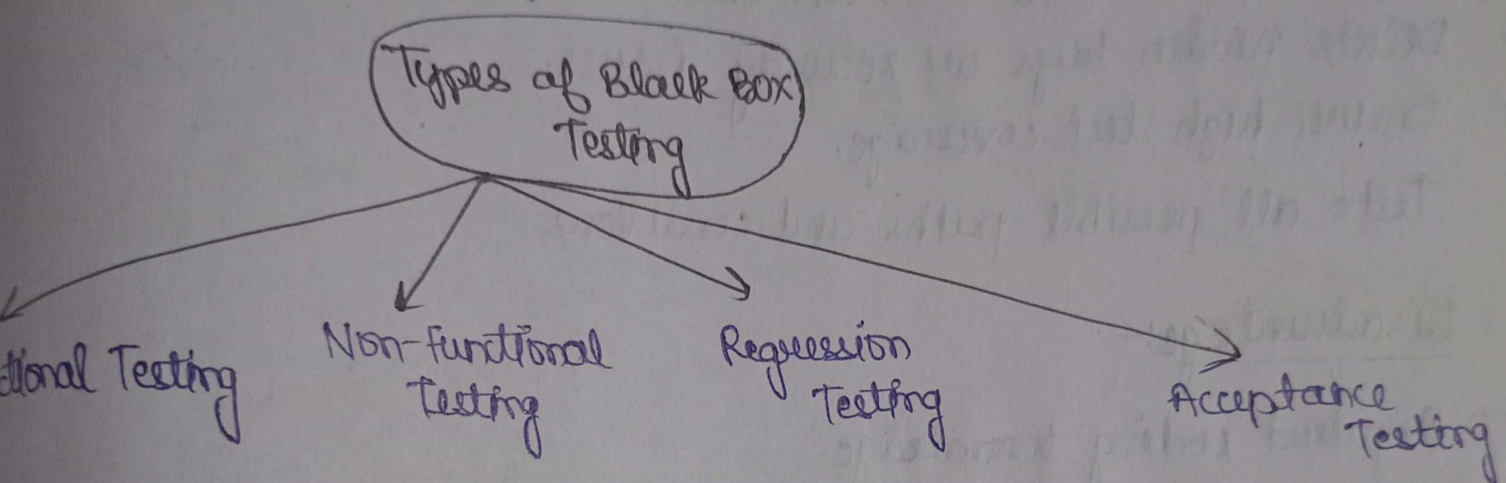
① Disadvantages:-

1. Doesn't test internal code quality.

might miss some internal errors.

Common Black Box Testing Techniques

Equivalence Partitioning.
Boundary Value Analysis.
Decision Table Testing.
State Transition Testing.



White Box Testing

Here we go inside a system and check ~~system~~ how actual function-ality is performed (all we making the product right).

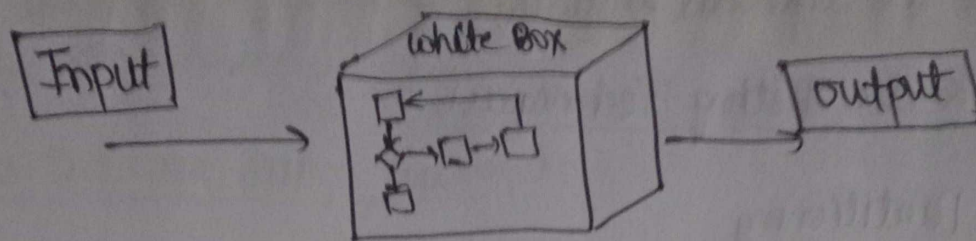
OR

White Box testing (also called clear box, open box, or glass box testing) is a method of testing where you look inside the code and test its internal structure, logic, and flow.

White box testing can be applied at the unit, integration and system levels of the software testing process.

+ Imagine opening a clock to see all the gears and testing how they move.

Usually done by developers or technical testers who understand



① Advantages

1. Helps optimize and clean the code.
2. Detects hidden bugs or security issues.
3. Ensure high test coverage.
4. Tests all possible paths and conditions.

① Disadvantage:-

1. Required coding knowledge.
2. Not suitable for user-level testing.

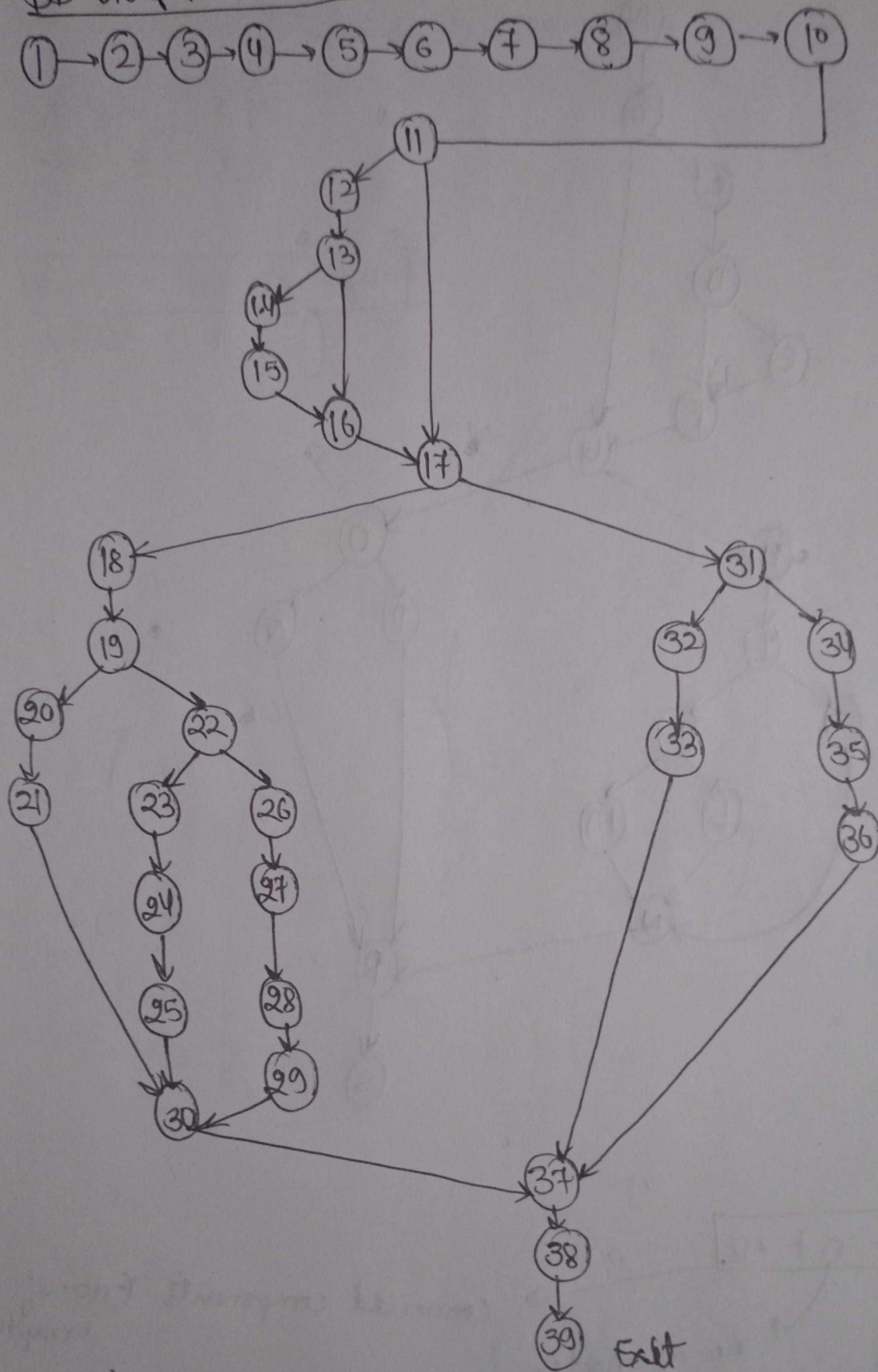
① Common White Box Testing Techniques

1. Statement Coverage.
2. Branch Coverage.
3. Path coverage.
4. Loop Testing.

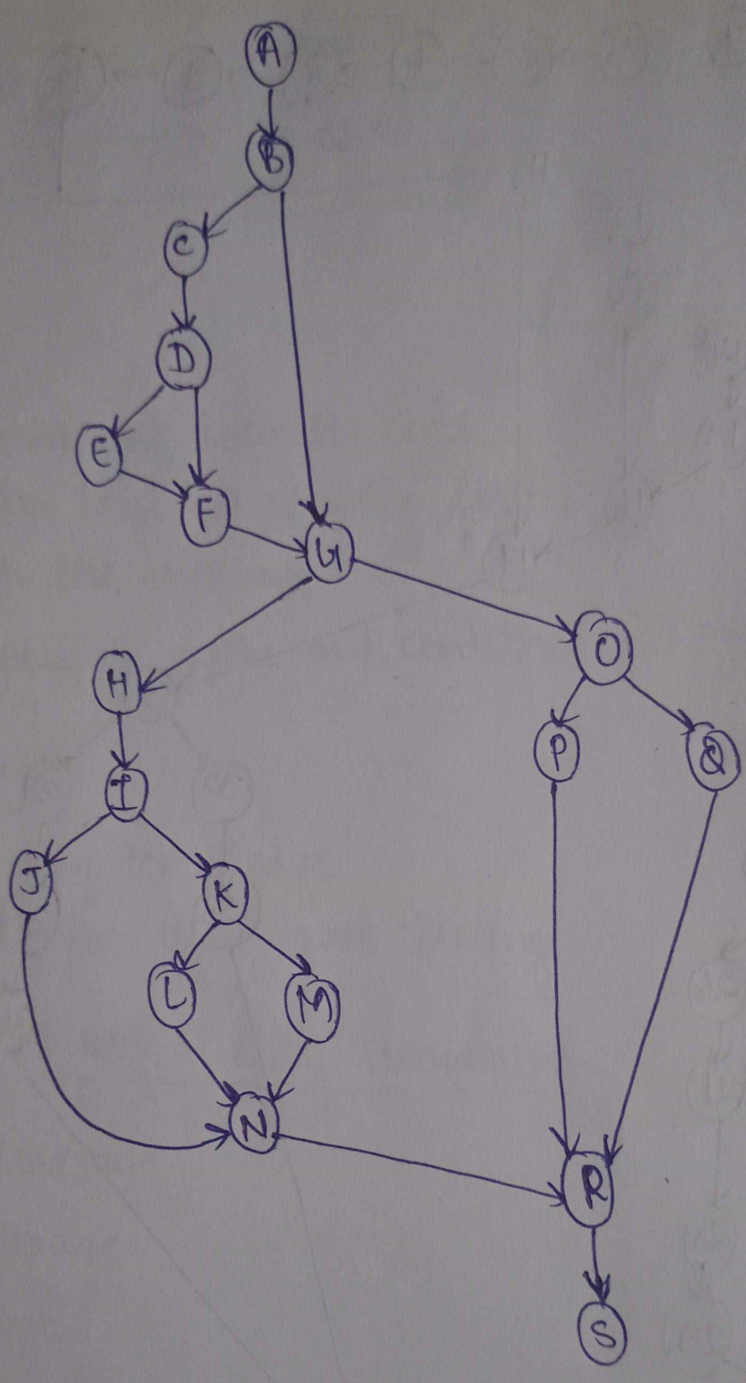
① Types of white box testing

1. Unit Testing.
2. Integration Testing.
3. Control flow Testing.
4. Branch Testing.

Ques: DD Graph Numerical:-



∴ DD path graph



Formula.

1.
$$V(G) = e - n + 2p$$

$\swarrow$ no. of edges $\searrow$ no. of nodes

connected components (no. of graphs)

$\Rightarrow V(G) = 24 - 19 + 2 \times 1$
 $\Rightarrow V(G) = 7$

② $V(n) = n+1$ $\rightarrow$ no. of decision nodes.

$\Rightarrow V(n) = n+1$

$\Rightarrow V(n) = n$

③ $V(n) = \text{No. of Regions}$

$\Rightarrow V(n) = n$

Ques 2

① PERT CHART (program evaluation and Review technique) :-

- ① It is a project management technique used to plan schedule and manage task with in a project specially those with a high degree of uncertainty in task duration.

⇒ Task Identification -

$$[O + m + P] / 6$$

O - optimistic.

m - most likely

P - pessimistic

$$\Rightarrow \frac{O + 4m + P}{6} \quad \therefore \text{Formula.}$$

Ques! If a task has an optimistic time 30 min, $P = 60$ min and most likely time 45 min. Calculate expected time.

$$\text{Soln} \Rightarrow \frac{30 + 4 \times 45 + 60}{6}$$

$$\Rightarrow \boxed{45 \text{ min.}}$$

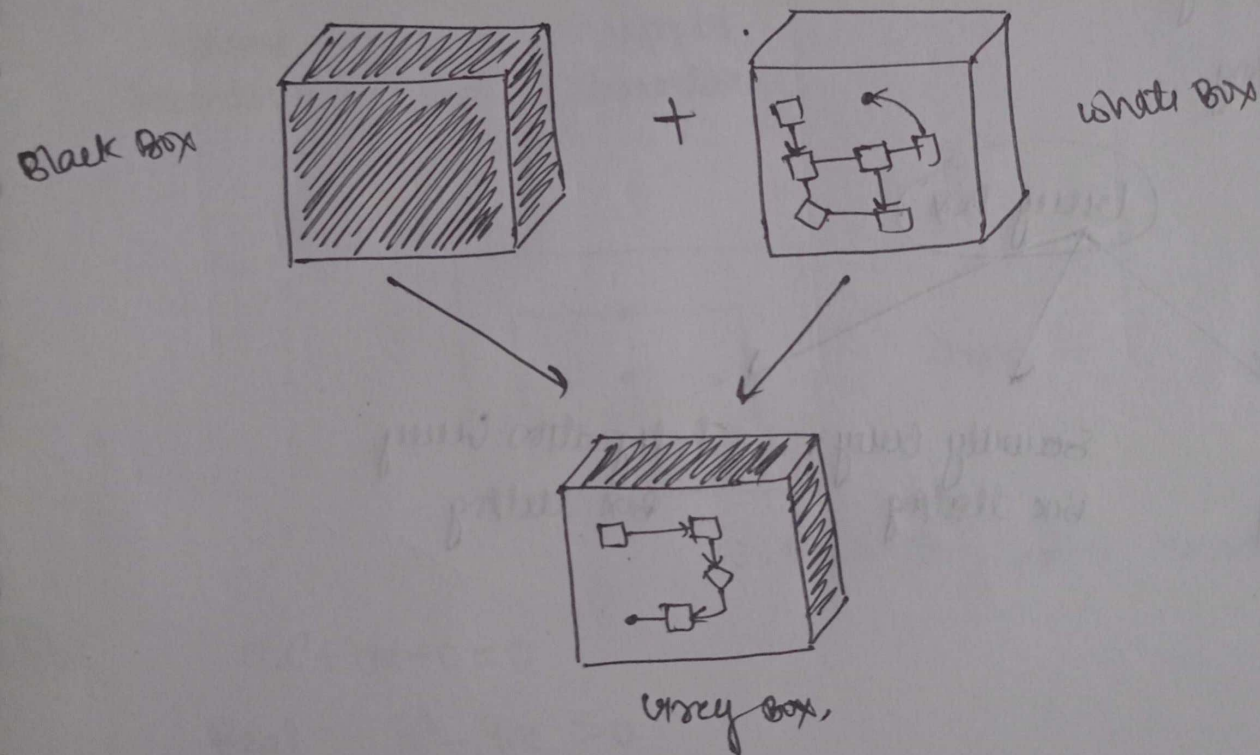
① Grey Box Testing

① Grey Box Testing is a type of software testing where the tester has partial knowledge of internal working of the system (not completely like white-box and not fully hidden like black-box).

→ It's a mix of white box and Black box.

Ex- Login system testing

- ↳ Tester knows that login connects to a database.
- ↳ Tests for valid/invalid login and possible sql injection.



① Advantages

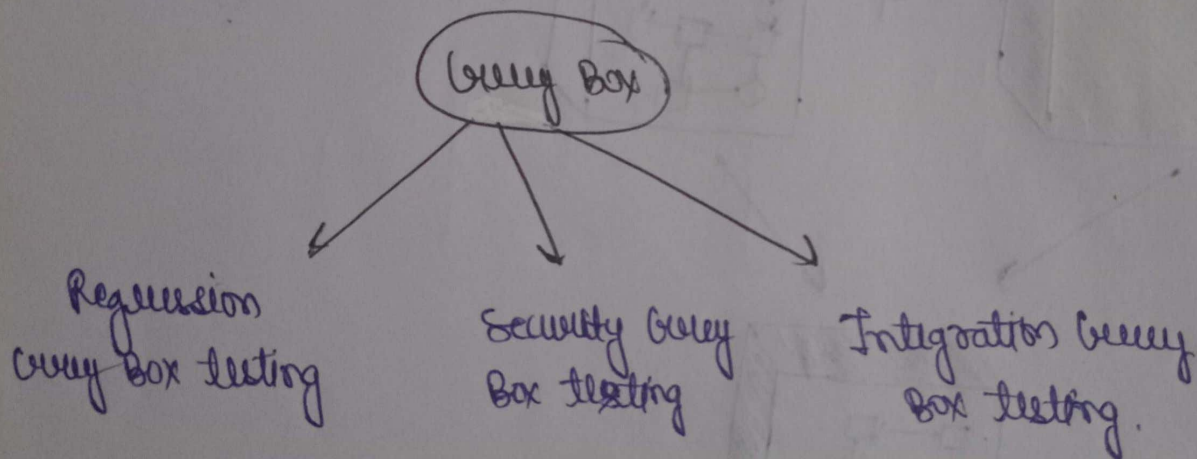
1. Better test coverage than black box.
2. Can be used when white-box access is limited.
3. Useful in integration and security testing.
4. Finds hidden bugs in data handling or logic.

① Disadvantages

1. Limited code knowledge.
2. May miss low level bugs.
3. Required technical + functional skills.

② Common Grey Box testing techniques

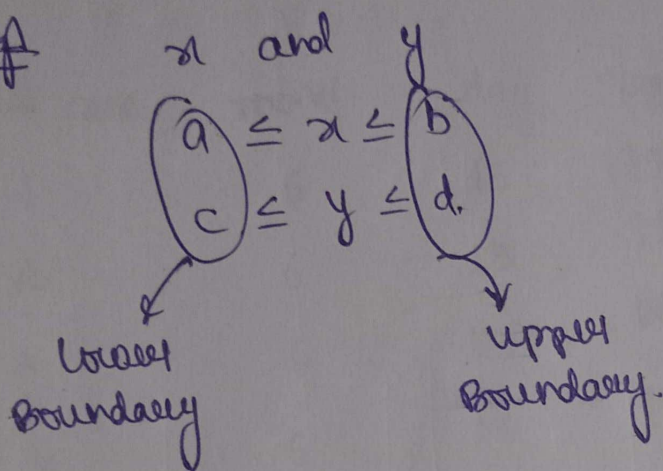
1. Matrix testing.
2. Regression Testing.
3. Security testing
4. Pattern testing



① Boundary value Analysis

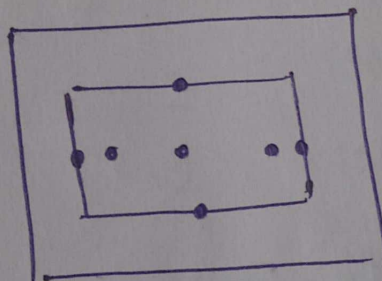
- ② The black box testing techniques are helpful for detecting any errors or threats that happened at the boundary values of valid or invalid partitions rather than focusing on the center of the input data.

Eg



Eg

1 to 100
(1, 100)



$(4n+1)$

∴ Boundary value analysis

Ques.

$$ax^2 + bx + c = 0$$

$$\text{Real} = b^2 - 4ac > 0$$

$$\text{Imaginary} = b^2 - 4ac < 0$$

$$\text{Equal} = b^2 - 4ac = 0$$

Ques 9. Consider a program for determining the previous date. Input Day, month, year with valid ranges as-

$$1 \leq \text{month} \leq 12$$

$$1 \leq \text{Day} \leq 31$$

$$1900 \leq \text{year} \leq 2000$$

Example

14 June 1900

Previous Date

Invalid Date

Test case	month	day	Year	output
1	6	15	1900	14 June 1900
2	6	15	1901	14 June 1901
3	6	15	1960	14 June 1960
4	6	15	1999	14 June 1999
5	6	15	2000	14 June 2000
6	6	1	1960	31 May 1960
7	6	2	1960	1 June 1960
8	6	30	1960	29 June 1960
9	6	31	1960	15 June 1960 Invalid
10	1	15	1960	14 Jan 1960
11	2	15	1960	14 Feb 1960
12	11	15	1960	14 Nov 1960
13	12	15	1960	14 Dec 1960

$$\Rightarrow 4n+1$$

$$\Rightarrow n = \text{no. of left cases.}$$

$$\Rightarrow 4 \times 3 + 1$$

$$\Rightarrow (13)$$

Hence proved.

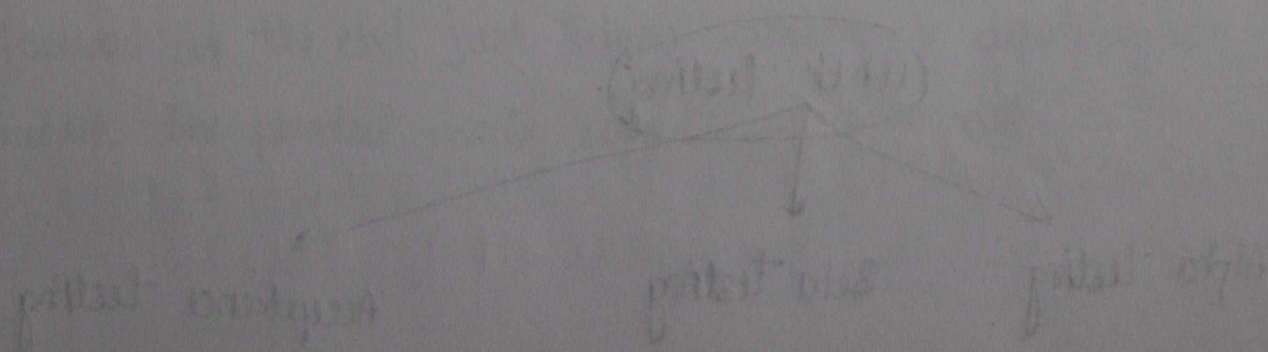
⑤ Difference Between Verification and Validation:-

⑤ Verification:-

1. Are you building it Right?
2. Check whether an artifact confirms its previous artifact.
3. Done by Developers.
4. Concurred with phase containment of errors.
5. Methods involves Review, Inspection, unit testing and
6. integration testing.
6. static and dynamic Activities

⑤ Validation:-

1. Have you build the Right thing?
2. Check the final product against specification.
3. Done by Testers.
4. Aim is to make final product error free.
5. Involves system testing.
6. only Dynamic.



① Unit Testing

① Unit testing is the process of checking small pieces of code to ensure that the individual parts of a program work properly on their own.

- Unit tests are used to test individual block (units) of functionality.
- Unit testing is done by developers.
- First level of software testing.

Ex:- The car manufacturing analogy

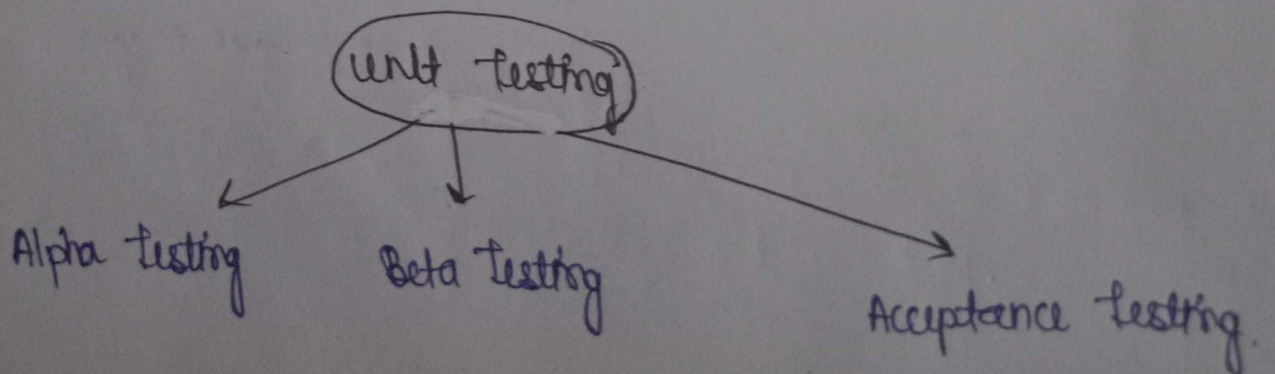
→ Different system developed separately

- ↳ Engine.
- ↳ Braking system.
- ↳ Fuel system.
- ↳ Ignition system.

② Advantages

1. Catches Bugs Early
2. Improves code Quality.
3. Support code changes
4. Speed up Development

③



1. Alpha testing:-

- ① Alpha testing is a type of testing performed by internal developers or testers before the product is released to actual users.

(OR)

- ① Any type of testing which is done on developer's side is called alpha testing, usually performed with artificial test cases.

Ex- A case is tested inside the company.

2. Beta testing:-

- ① Beta testing is a type of software testing where the product is released to real users outside the company to use (a limited group) and provide feedback for final release.

(OR)

- ① Any type of testing which is done at customer's side is called beta testing, usually performed with real time data.

Ex- A case is tested in different real conditions / areas.

3. Acceptance testing:-

- ① Acceptance testing is the final test done before a software is released to check if it works the way the user or client wants.

→ Performed at the end just before release on development

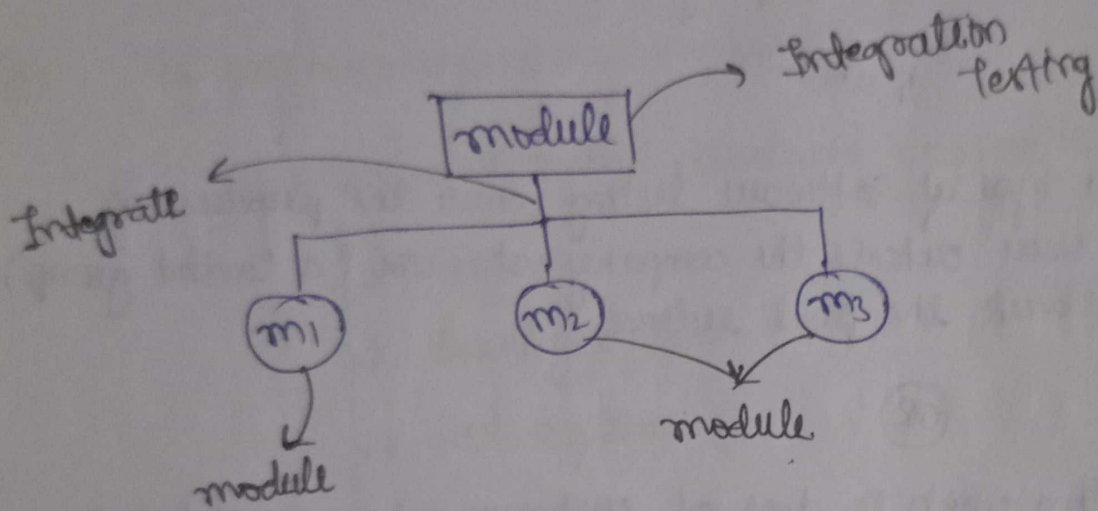
→ To check the product meets requirements or not.

① Integration testing

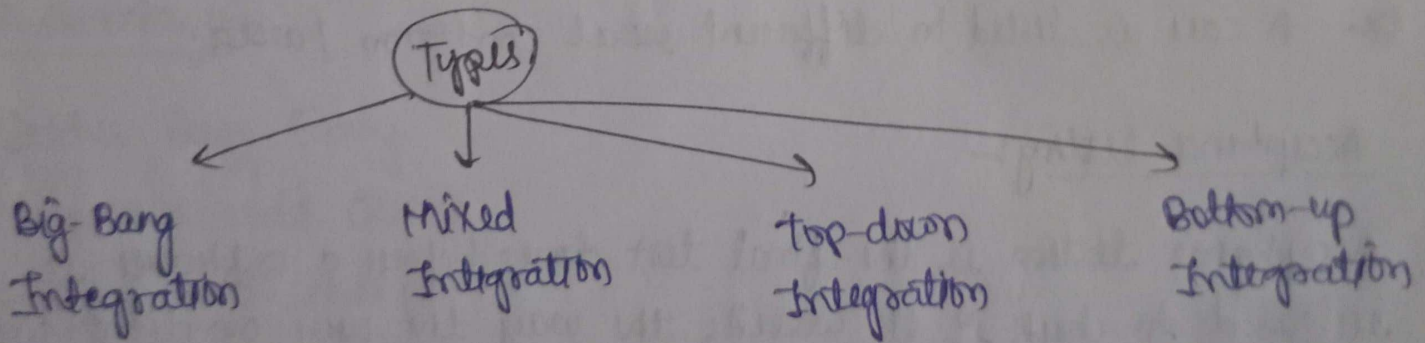
② Integration testing is conducted to evaluate the complianes of a system or component with specified functional requirements.

(OR)

③ Integration testing checks if two or more parts of program work well together.



Ex: Book search service:



1. Big-Bang Integration

② In this testing all parts of the software are put together at once, and then you test the whole system all together.

Ex:- Test whole can system together.

2. Top-down

- ① In this testing you start testing from top (user interface) to the lower part (backend and database).

3. Bottom-up

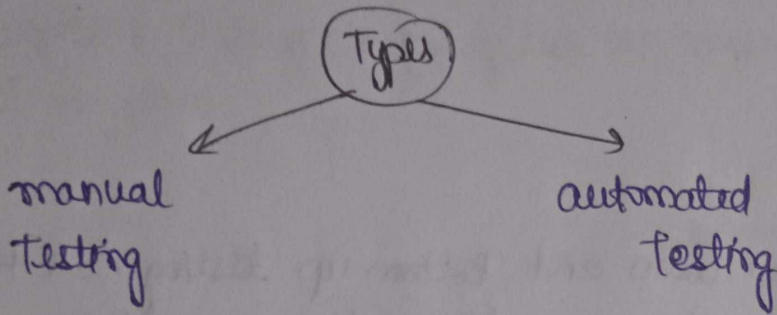
- ① vice-versa.

4. Mixed Integration

- ① This testing combines Top down and Bottom up testing, we test top and bottom part at the same time, then connect everything together.

① System testing:-

- ① System testing is a level of testing that validates the complete and fully integrated software project. The purpose of system test is to evaluate the end-to-end system specification.



1. Manual testing

- ① In manual testing, a person tester checks the software manually by using it, without any tools or automation.

Ex: Calculator:-

→ you open the app

→ you press button to add, subtract, and check if the answers are correct

2. Automated testing

- ① In automated testing you use a computer program to automatically test the software instead of doing it by hand or manually.

Ex: sum() (program).

① Introduction to Automation testing and testing tools.

② Automated testing process

- ③ It is a technique where the tester writes scripts independently and use suitable software or Automation Tools to test the software.

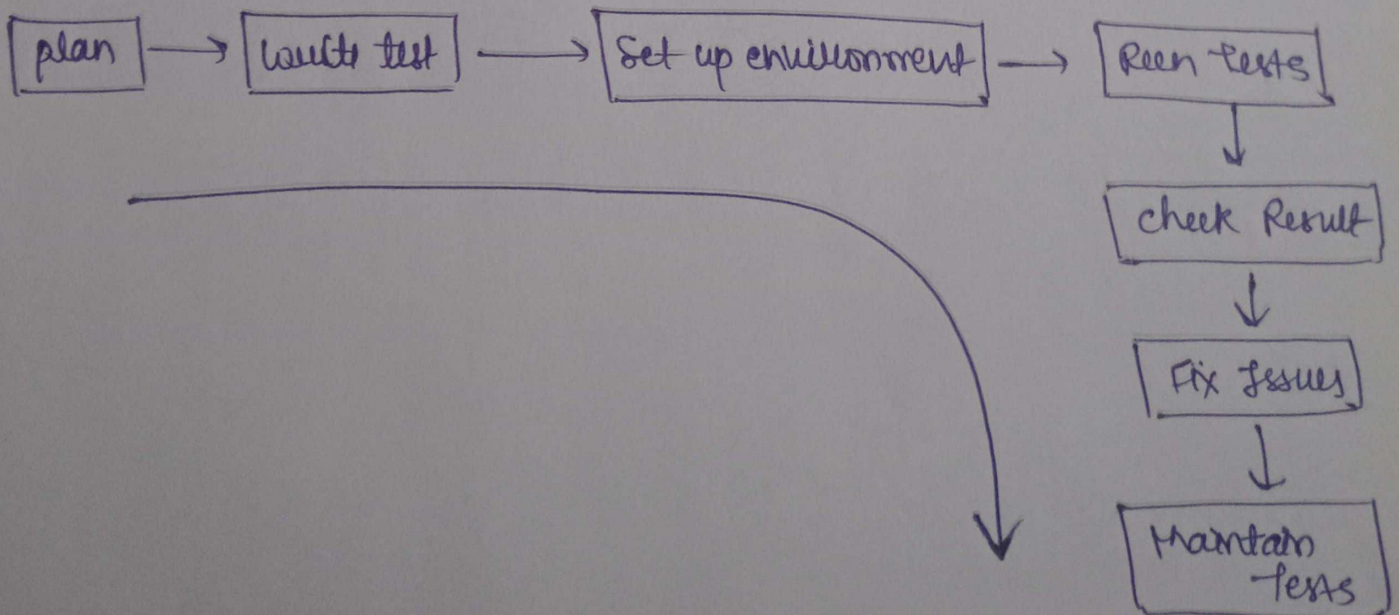
(OR)

- ④ Automated testing means using software tests to run tests on your code automatically, without doing it by hand or manually.

⑤ Advantages

1. Saves time.
2. Finds bugs early
3. Runs tests faster and more often
4. Great for big projects.

⑥ Steps



① Types of Automated Testing Frameworks

1. Linear Framework.
2. Modular Framework.
3. Data-Driven Framework.
4. Keyword-Driven Framework.
5. Hybrid Framework.

② Tools for Automated Testing

1. Selenium.
2. Appium.
3. Katalon.
4. Ranorex Studio.
5. SmartBear TestComplete.